



Jomo Kenyatta University of Agriculture and Technology

College of Engineering and Technology

School of Mechanical, Materials, and Manufacturing Engineering

Department of Mechatronic Engineering

Modification and Improvement of a Coffee Berry Sorting Machine

(FYP 22-22)

Interim Report

Joe Annel (ENM221-0051/2021)

Prosper Muuo (ENM221-0218/2021)

Supervisor

Ms Maureen Andanje

Dr. Maurine Andanje

February 27, 2026

Declaration

We, the undersigned, declare that this Interim Report is our original work undertaken in partial fulfillment of the requirements for the award of the Bachelor of Science degree in Mechatronic Engineering from Jomo Kenyatta University of Agriculture and Technology. This report has not been submitted for any degree or examination in any other institution.

Joe Annel

ENM221-0051/2021

Signature: _____

Date: _____

Prosper Muuo

ENM221-0218/2021

Signature: _____

Date: _____

This Interim Report has been submitted with my approval as University Supervisor:

Ms Maureen Andanje **Dr. Maurine Andanje**

Department of Mechatronic Engineering

Jomo Kenyatta University of Agriculture and Technology

Signature: _____

Date: _____

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Ms Maureen Andanje, for her invaluable guidance, encouragement, and technical insights throughout this project. Her constructive feedback has been instrumental in shaping both the direction and quality of this work.

We also extend our appreciation to the Department of Mechatronic Engineering at Jomo Kenyatta University of Agriculture and Technology for providing the resources and facilities necessary for conducting this research.

Special thanks to the previous group whose foundational work on the coffee berry sorting machine provided the starting point for our modifications and improvements.

Finally, we acknowledge the support of our families and colleagues who provided encouragement throughout the course of this project.

Abstract

Automated coffee berry sorting is a critical step in improving the quality and efficiency of coffee processing, particularly for small- and medium-scale producers in Kenya. This project addresses the limitations of an existing image-based sorting machine developed by fifth-year mechatronics students at JKUAT, which employed a Raspberry Pi vision system and air-ejection actuation. Key challenges identified include processing latency, motion blur under dynamic conveyor conditions, and insufficient classification throughput.

This interim report presents the work completed to date, covering a review of relevant literature on computer vision-based sorting systems, the proposed methodology for improving the system, and progress achieved. A custom YOLOv11 object detection model has been trained on a dataset of harvested coffee berry images, targeting four ripeness classes: unripe, semi-ripe, ripe, and overripe. Initial evaluation results demonstrate promising detection performance, with the model achieving competitive mean average precision (mAP) values at moderate inference speeds.

The remaining work focuses on integrating the trained model with the physical sorting machine, validating real-time performance on the conveyor, and optimizing the detection-to-actuation pipeline for improved throughput and reduced latency. A detailed work plan for the completion of remaining objectives is also presented.

Keywords: Coffee berry sorting, YOLOv11, computer vision, conveyor, real-time detection, Raspberry Pi, mechatronics

This is not a research/conference paper. Hence remove the keywords.

Contents

Declaration	I
Acknowledgments	II
Abstract	III
1 Introduction	1
1.1 Background of Study	1
1.2 Problem Statement	2
1.3 Objectives	3
1.3.1 Main Objective	3
1.3.2 Specific Objectives	3
1.4 Justification	4
1.5 Scope of the Project	4
1.6 Expected Outcomes	5
2 Literature Review	6
2.1 The Reference Project: Previous Coffee Berry Sorting Machine	7
2.1.1 System Architecture of the Reference Machine	7
2.1.2 Identified Limitations of the Reference Machine	8
2.2 Challenges of Manual and Early Automated Sorting	10
2.3 Mechanical Material Handling	11
2.4 Camera Technology: Rolling Shutter vs. Global Shutter	13
2.5 Deep Learning Object Detection for Sorting	14
2.5.1 CNN-Based Feature Extraction	14
2.5.2 MobileNetV2 and its Limitations	14
2.5.3 YOLO Family: Real-Time Detection with Localisation	15
2.5.4 YOLOv11 on Embedded Platforms	17
2.6 Binary (Accept/Reject) vs. Fine-Grained Classification	17

2.7	Embedded Co-Processor Architectures	19
2.8	Summary of Literature Gaps Addressed by This Project	20
3	Methodology	22
3.1	Performance Analysis of Existing Machine	22
3.2	Dataset Collection and Model Training	23
3.2.1	Image Acquisition	23
3.2.2	Annotation	24
3.2.3	Dataset Split	25
3.3	YOLOv11 Model Training	25
3.3.1	Framework and Backbone	25
3.3.2	Training Configuration	26
3.3.3	Evaluation Metrics	26
3.4	Sorting Mechanism Improvements	26
3.4.1	Hardware Platform	27
3.4.2	Detection-to-Actuation Pipeline	30
3.4.3	ESP32 Pin Assignment and Wiring	33
3.4.4	Transistor Technology Selection: BJT vs. Logic-Level MOSFET	35
3.4.5	Solenoid Driver Circuit	36
3.5	Circuit Simulation in Proteus	37
3.5.1	Simulation Scope	37
3.5.2	Simulation Methodology	38
3.6	Validation Strategy	39
4	Results and Discussion	40
4.1	Overview	40
4.2	Performance Analysis of Existing System	40
4.3	Sorting Mechanism Improvements	41
4.3.1	Procured and Verified Hardware	41

4.3.2	Proteus Circuit Simulation Results	43
4.4	Hardware Upgrade and Model Development	46
4.4.1	Dataset and Class Reduction	46
4.4.2	YOLOv11n Training Results	47
4.4.3	Dataset Visualisations	50
4.4.4	Sample Detection Output	52
4.5	Software Development	53
4.6	Discussion: Progress Against Objectives	56
5	Conclusion	57
5.1	Summary	57
5.2	Future Work	58
5.3	Project Schedule	58
5.4	Risk Assessment	58
A	Circuit Schematics	61
B	Schedule and Budget	64
C	Dataset and Model Outputs	65
D	Mechanical Design Schematics	66

List of Figures

1.1	Harvested coffee berry variation showing visual diversity in colour, size, and surface texture	2
2.1	Ripe coffee berries showing colour variation from green to deep red	6
2.2	Reference model validation predictions showing four-class classification (un-ripe, semi-ripe, ripe, overripe) with visible confusion between visually similar categories	9
2.3	Reference model F1-score performance across the four ripeness categories, highlighting the classification difficulty at the semi-ripe/ripe boundary . . .	10
2.4	Comparison of manual and electronic coffee sorting methods	11
2.5	Fully fabricated mechanical assembly from the reference implementation [1], showing integrated hopper, vibrating feeder, chute, and frame structure. This mechanical subsystem forms the foundation upon which the improved imaging and control systems are built.	12
2.6	CAD drawing of the air-ejector nozzle assembly from the reference sorting machine showing solenoid valve, nozzle orientation, and berry deflection trajectory [1]	13
2.7	Image processing pipeline for YOLOv11-based sorter on Raspberry Pi 4: frame capture (global shutter) → pre-processing → YOLOv11 inference → classification (accept/reject) → ejection control	16
2.8	Overview of classification approaches considered: classical machine learning, CNN-based models, and single-stage detectors (YOLO family)	17
2.9	Visual comparison of ripe (accept) and unripe/semi-ripe (reject) coffee berries. The binary classification boundary maps to the deep-red colour transition.	18
2.10	Proposed co-processing architecture: Raspberry Pi 4 handles inference; ESP32-WROOM-32 manages all hard real-time actuation via UART. . . .	20

3.1	Overall architecture of the improved coffee berry sorting machine: global-shutter camera, Raspberry Pi 4 inference engine, ESP32 real-time co-processor, and MOSFET solenoid driver stages	22
3.2	Co-processing architecture: Raspberry Pi 4 handles image processing and YOLOv11 classification; ESP32 handles motor control, actuator interfacing, and timing	28
3.3	System control flow: task distribution between Raspberry Pi 4 (inference and tracking) and ESP32 (actuation, timing, and PWM control)	29
3.4	Image processing pipeline for the YOLOv11-based sorter: frame capture (global shutter) → pre-processing → YOLOv11 inference on Raspberry Pi 4 → tracking → UART command to ESP32	31
3.5	UART communication and data flow between Raspberry Pi and ESP32	32
3.6	Fritzing circuit diagram of the ESP32 solenoid driver implementation: IRLZ44N MOSFET switching stages, flyback diodes, gate resistors, and Raspberry Pi UART connection	37
4.1	Electrical schematic: power distribution, processor connections, actuator outputs, and communication links	42
4.2	System block diagram: processor hierarchy, communication links, and actuator outputs with power distribution	43
4.3	Complete Proteus ISIS simulation: ESP32 with four IRLZ44N solenoid driver channels, vibrating motor PWM, LED strip driver, 2-channel optocoupler relay, and UART virtual terminal	44
4.4	Proteus oscilloscope: MOSFET turn-ON — gate rises to 3.3 V and drain falls from 12 V to ≈ 0 V with no ringing (IRLZ44N, 220 Ω gate resistor, 100 k Ω pull-down)	45
4.5	Proteus oscilloscope: MOSFET turn-OFF — drain recovers to 12 V; 1N5819 clamps inductive kickback spike within 0.5 V of the supply rail	45
4.6	YOLOv11n comprehensive training metrics over 100 epochs showing convergence of loss, precision, recall, and mAP on the binary coffee berry dataset	47

4.7	F1-score vs. confidence threshold curve for the YOLOv11n binary classifier, showing optimal operating point for accept/reject classification	48
4.8	Precision-Recall curve demonstrating the trade-off between precision and recall across confidence thresholds for both accept and reject classes	49
4.9	Normalised confusion matrix on the test split showing classification accuracy for accept (ripe) and reject (unripe/semi-ripe/overripe) categories . .	50
4.10	Ground truth label distribution visualisation showing the spatial distribution and class balance of accept and reject instances across the training dataset	51
4.11	Validation batch 0 with ground-truth bounding box annotations for accept (ripe) and reject (unripe/semi-ripe) berries	52
4.12	YOLOv11n inference output on validation batch 0: predicted accept/reject bounding boxes with confidence scores demonstrating the model's classification performance on unseen test images	53
4.13	System control flow: Pi 4 handles image processing and YOLOv11 inference; ESP32 handles motor control, timing, and actuation	55
A.1	Electrical schematic	61
A.2	ESP32 pin mapping schematic	62
A.3	Circuit implementation layout (Fritzing)	63
A.4	Proteus full-circuit simulation	63
B.1	Project schedule from the approved proposal	64
B.2	Project budget from the approved proposal	64
C.1	Reference model F1 curve	65
C.2	Reference model validation batch labels	66
D.1	Bottom plate technical drawing with dimensions and specifications	67
D.2	Chute assembly technical drawing with dimensions and specifications . . .	68
D.3	Chute mounting bracket technical drawing with dimensions and specifications	69
D.4	Vibrating feeder trough technical drawing with dimensions and specifications	70
D.5	Hopper assembly technical drawing with dimensions and specifications . .	71

List of Tables

2.1	Literature gaps and corresponding improvements in this project	21
3.1	Binary coffee berry classification scheme	25
3.2	YOLOv11 training hyperparameters	26
3.3	Raspberry Pi, ESP32, and Peripheral Wiring Assignment	34
3.4	BJT (PN2222A) vs. Logic-Level MOSFET (IRLZ44N) switching comparison	35
4.1	Measured baseline performance of existing sorting machine	40
4.2	Dataset split summary (2,000-image binary dataset)	47
4.3	YOLOv11n evaluation results on test split	50
4.4	Progress summary against specific objectives	56

1 Introduction

1.1 Background of Study Include citations to indicate the source of your background.

The global coffee industry remains a significant contributor to the economies of many developing countries, supporting millions of smallholder farmers worldwide. Coffee production involves several critical stages, from cultivation and harvesting to processing and packaging. Among these stages, the sorting of coffee berries is one of the most labor-intensive and quality-sensitive processes. Traditionally, coffee berry sorting has relied heavily on manual visual inspection, where workers separate berries based on ripeness, size, and visible defects. This approach is time-consuming, inconsistent, and costly, often leading to variations in coffee quality and increased operational expenses.

Manual sorting methods are highly susceptible to human error and fatigue, which can result in misclassification of berries and reduced overall product quality. Furthermore, the labor-intensive nature of manual sorting limits scalability and efficiency, particularly during peak harvest periods when labor shortages are common. These challenges highlight the need for automated sorting solutions capable of performing classification tasks with higher accuracy, consistency, and throughput.

Recent advancements in computer vision, machine learning, and embedded systems have enabled the development of automated coffee berry sorting machines that utilize image-based detection and real-time classification techniques. Previous work by fifth-year mechatronics engineering students resulted in the development of an image-based coffee berry sorting machine incorporating a vibrating conveyor system, optical detection, and air-ejection mechanisms controlled by a Raspberry Pi-based processing unit. While the system successfully demonstrated automated sorting, practical challenges were identified during real-time operation, particularly related to motion blur, processing latency, and limited computational throughput.



Figure 1.1: Harvested coffee berry variation showing visual diversity in colour, size, and surface texture

State the source of this image

The visual diversity in harvested coffee berries—including variations in colour, size, and surface texture, as illustrated in Figure 1.1—presents a significant challenge for accurate automated classification.

This project proposes the modification and improvement of the existing coffee berry sorting machine by addressing the identified limitations in image acquisition, processing speed, and real-time classification performance. By enhancing the vision processing pipeline, optimizing algorithms, and evaluating alternative hardware or processing architectures, the project aims to improve sorting accuracy and throughput while maintaining a cost-effective and scalable design.

1.2 Problem Statement

The quality of coffee berries at the processing stage directly determines the final product grade, market value, and competitiveness in both local and international markets. Despite

the successful demonstration of automated sorting in the previous system, it exhibits several critical technical limitations:

**Explain what motion blur and illumination inconsistencies are.
And why are they a limitation in sorting of the berries.**

1. **Motion blur and illumination inconsistency:** Dynamic motion of berries on the conveyor and variable ambient lighting introduce image degradation, reducing classification accuracy.
2. **Processing latency:** The limited computational capacity of the Raspberry Pi 3 causes delays across the frame acquisition, preprocessing, inference, and actuation pipeline, constraining throughput.
3. **Model accuracy and class confusion:** The original four-class model confused visually similar semi-ripe and ripe berries, causing commercially acceptable red berries to be rejected and semi-ripe berries to pass into the accept stream, reducing economic yield.

There is a clear need to improve the vision system, processing architecture, and real-time control strategy to achieve higher throughput, lower latency, and improved classification accuracy.

1.3 Objectives

1.3.1 Main Objective

To modify and improve an existing automated coffee berry sorting machine.

1.3.2 Specific Objectives

**Yur specific objectives should point towards
what needs to be done.**

1. To analyse the performance limitations of the existing image-based coffee berry sorting system, with emphasis on motion blur, processing latency, and classification accuracy.

2. **To improve on the sorting mechanism.** This objective is too vague. Ensure that it is very specific on what is to be done.
3. To evaluate alternative or upgraded processing hardware capable of supporting higher frame rates and faster inference.

1.4 Justification

Coffee remains one of Kenya's most valuable agricultural exports, and quality grading plays a major role in determining its economic value. Automated sorting systems offer a practical solution for improving quality consistency and reducing labour dependency. Improving the existing image-based system provides a cost-effective pathway by building upon already validated concepts while addressing observed technical shortcomings.

The proposed improvements are expected to:

1. Enhance accuracy and throughput in coffee berry sorting.
2. Enable better utilisation of affordable embedded vision technologies.
3. Increase the feasibility of locally developed agri-tech solutions.
4. Contribute practical learning and innovation in mechatronics system integration.

1.5 Scope of the Project

This project is limited to the modification and improvement of the existing coffee berry sorting machine. Specifically:

- The mechanical conveyor and air-ejection hardware remain unchanged from the original design unless critical faults are identified.
- Focus is placed on the vision and control sub-systems: camera selection, lighting, model training, inference pipeline, and microcontroller–Pi communication.

- Classification uses a binary **ACCEPT/REJECT** scheme: the **accept** class covers uniform deep-red ripe berries; the **reject** class covers unripe (green), semi-ripe (yellow/orange), overripe (dark/black), and physically damaged berries. This aligns with Kenyan cooperative grading practice.
- Deployment targets the Raspberry Pi 4 or equivalent single-board computer for embedded inference.

1.6 Expected Outcomes

Your expected outcomes should link to what you have stated in the specific objectives.

Upon completion of this project, the following outcomes are expected:

1. A YOLOv11-based coffee berry detection model achieving binary classification accuracy >90% (mAP@0.5) on the **accept/reject** scheme, replacing the existing MobileNetV2 classifier.
2. An end-to-end sorting pipeline latency of less than 100ms (detection to solenoid trigger), achieved through the Pi 4 + ESP32 co-processing architecture.
3. Sorting throughput exceeding 150 berries per minute, an improvement over the baseline of approximately 35 berries per minute.
4. A validated Proteus circuit simulation of the ESP32 control circuit prior to physical implementation, confirming correct solenoid switching, UART timing, and motor PWM behaviour.
5. A fully characterised and validated sorting system with documented design drawings, test results, and a reproducible codebase.

Note: These expected outcomes are defined here, prior to system implementation. Actual results and performance comparisons are presented in Chapter 4.

2 Literature Review

The global coffee industry plays a critical role in the economies of many developing countries, including Kenya, where it provides livelihoods to millions of smallholder farmers. Coffee processing involves several critical stages, among which the sorting of coffee berries determines final product quality and market value. The presence of unripe, overripe, or defective berries significantly degrades cup quality, leading to price penalties and rejection by premium buyers [2].

Automated sorting systems based on machine vision offer a path to consistent, high-throughput grading. A previous fifth-year project at this institution developed an image-based coffee berry sorting machine that demonstrated the feasibility of automated sorting but revealed significant technical gaps in real-time performance. This review surveys the relevant literature, documents the previous system and its known limitations, and identifies the specific gaps addressed by this work.



Figure 2.1: Ripe coffee berries showing colour variation from green to deep red

Provide the source for this image.

2.1 The Reference Project: Previous Coffee Berry Sorting Machine

Change this heading to be "Previous Works"
then have a sub-section for the specific project

The foundation of this work is a fifth-year mechatronics engineering project that developed an automated coffee berry sorting machine [1]. That system established the core mechanical and electronic architecture on which this project builds.

Ensure your citations are sequential

2.1.1 System Architecture of the Reference Machine

The reference system integrated four main subsystems:

1. **Mechanical handling:** A vibrating eccentric-motor tray singulated berries from a hopper onto an inclined chute. Gravity and vibration aligned berries into a near-single-file stream before the imaging zone.
2. **Image acquisition:** A Raspberry Pi Camera Module V2 (rolling shutter, 8MP Sony IMX219) captured frames above the chute at fixed intervals.
3. **Classification:** A MobileNetV2-derived model [3], deployed as a TensorFlow Lite INT8 quantised file on the Raspberry Pi 3B+, performed per-frame berry classification.
4. **Actuation:** Compressed-air solenoid jets, triggered by the Pi's GPIO pins, deflected reject-class berries into separate chutes; accepted berries continued to the accept bin under gravity.

The mechanical chute and vibrating feeder proved robust and reliable in field testing. This project retains that mechanical subsystem unchanged, focusing improvements exclusively on the electronics, vision, and software layers.

2.1.2 Identified Limitations of the Reference Machine

Post-build evaluation of the reference machine identified three critical performance bottlenecks [1]:

1. **Motion blur:** The rolling-shutter sensor read out rows sequentially over a ~ 30 ms window. At operating chute speeds of $0.4\text{--}0.6\text{ m s}^{-1}$, berries translated 12–18 mm per frame, producing visible smearing that degraded classification confidence, particularly at the semi-ripe/ripe colour boundary.
2. **Processing latency:** Running MobileNetV2 inference, GPIO triggering, and camera I/O on a single Raspberry Pi 3B+ core produced end-to-end latency of 280–400 ms per berry. This limited practical throughput to approximately 35 berries per minute—far below the commercial target of >150 per minute.
3. **Fine-grained class confusion:** The four-class model (unripe, semi-ripe, ripe, overripe) struggled with the visually similar semi-ripe and ripe classes. This confusion produced mis-ejections: commercially acceptable red berries were rejected and semi-ripe berries were accepted into the output stream, reducing economic yield.

Figures 2.2 and 2.3 illustrate the performance characteristics of the reference four-class MobileNetV2 model, showing the classification challenges across the four categories.



Figure 2.2: Reference model validation predictions showing four-class classification (unripe, semi-ripe, ripe, overripe) with visible confusion between visually similar categories

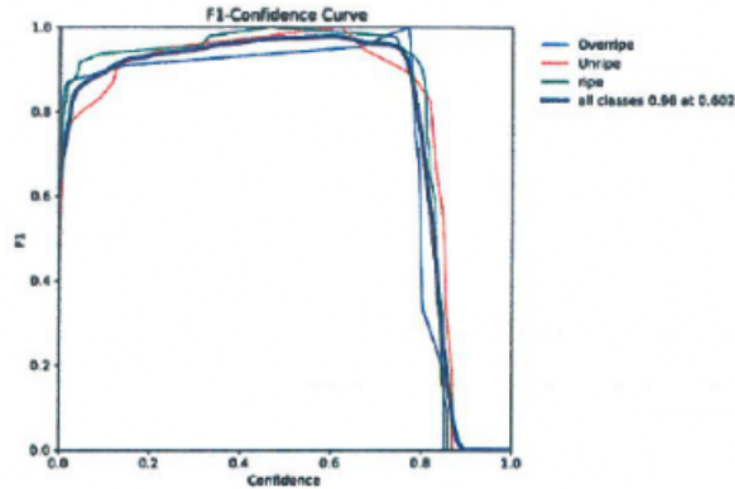


Figure 4.13: F1 score

Figure 2.3: Reference model F1-score performance across the four ripeness categories, highlighting the classification difficulty at the semi-ripe/ripe boundary

2.2 Challenges of Manual and Early Automated Sorting

1. Labour Intensity and Fatigue: Prolonged visual inspection leads to fatigue, reducing sorting accuracy and consistency [2]. Fatigued workers are more likely to misclassify berries, allowing defective berries to pass or rejecting acceptable ones.

2. Human Error and Subjectivity: Significant variability in sorting outcomes has been observed across different shifts, even when standardised grading guidelines were provided [4]. This subjectivity results in inconsistent quality between batches.

3. Limited Throughput: Human sorting speed becomes a bottleneck during peak harvest seasons [5]. Automated systems address this but introduce the challenges of motion blur, latency, and fine-grained class confusion documented above.

2.3 Mechanical Material Handling

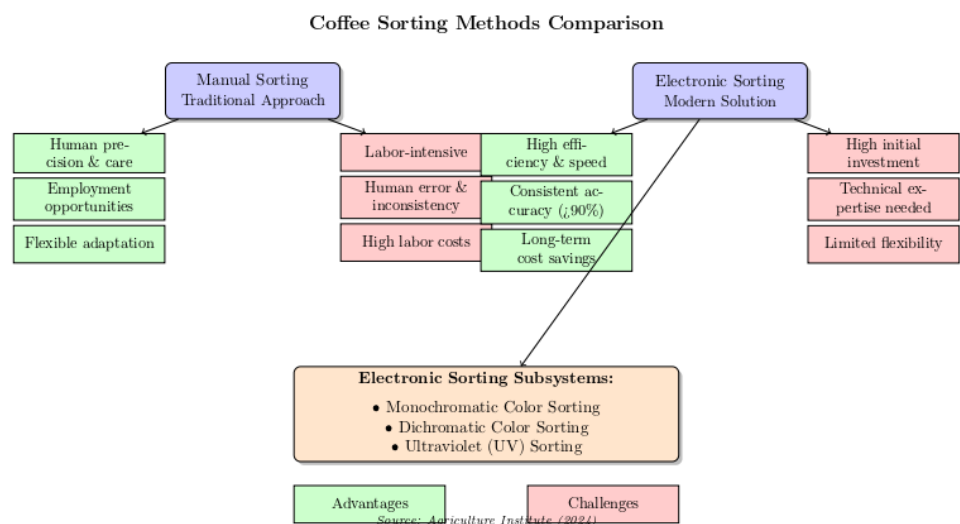


Figure 2.4: Comparison of manual and electronic coffee sorting methods

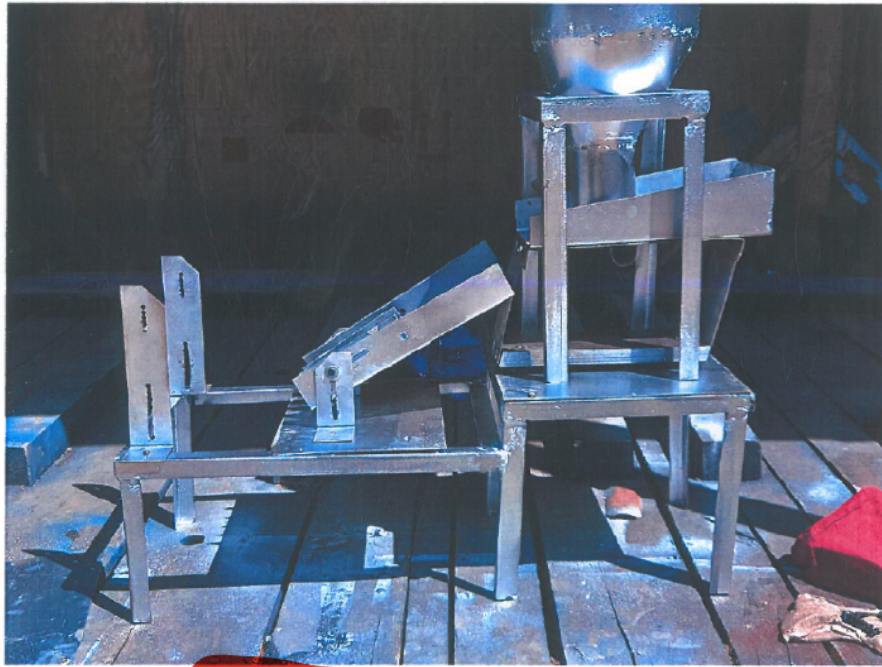


Figure 3.16: Fabricated mechanical assembly

Check the sequence of your figures

Figure 2.5: Fully fabricated mechanical assembly from the reference implementation [1], showing integrated hopper, vibrating feeder, chute, and frame structure. This mechanical subsystem forms the foundation upon which the improved imaging and control systems are built.

Vibrating feeder-chute systems are standard in agricultural sorting. The amplitude and frequency of an eccentric motor are tuned to prevent bridging while maintaining adequate throughput and berry singulation. Chute inclination angle and surface finish further influence flow rate and separation. Literature confirms that this approach produces reliably consistent berry spacing at throughputs of 100–300 berries per minute in laboratory prototypes [6].

The air-jet deflection mechanism, which the reference system uses for separation, operates on a detect-decide-deflect principle. Berries passing the imaging zone trigger compressed-air jets timed to deflect reject-class berries into separate chutes while accepted berries continue by gravity. Key advantages include minimal physical contact and high speed; limita-

tions include compressed-air infrastructure requirements and tight timing constraints [7].

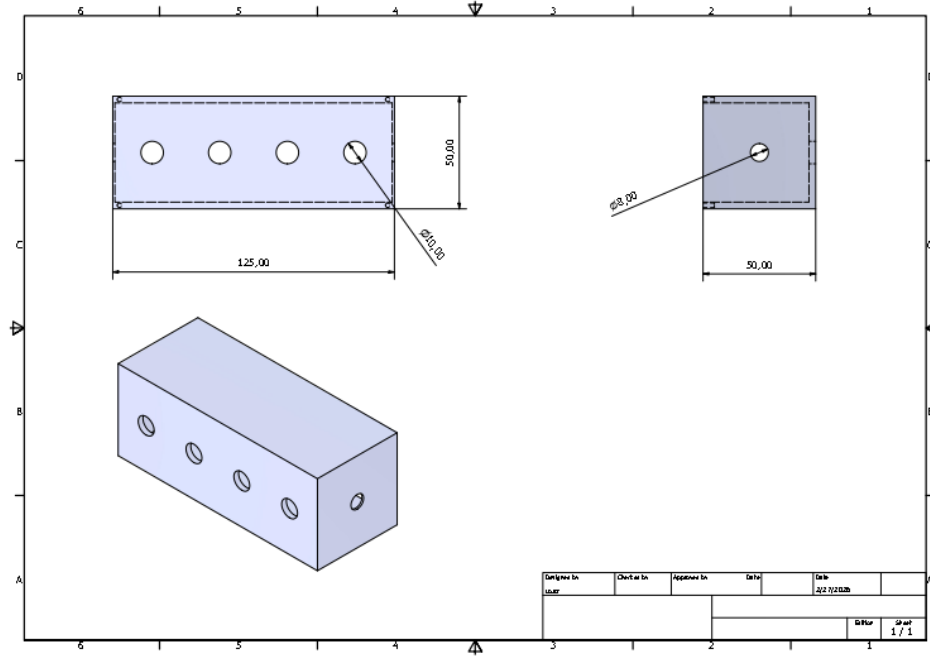


Figure 2.6: CAD drawing of the air-ejector nozzle assembly from the reference sorting machine showing solenoid valve, nozzle orientation, and berry deflection trajectory [1]

2.4 Camera Technology: Rolling Shutter vs. Global Shutter

Conventional CMOS cameras read out pixel rows sequentially (rolling shutter). For static scenes this is adequate, but when objects move faster than the sensor readout time, each row captures a slightly different object position, producing a characteristic shear distortion (jello effect) and edge smearing [8]. In the reference machine this was identified as the primary source of image degradation.

A **global-shutter** sensor exposes all pixels simultaneously, eliminating motion-induced distortion entirely. Global-shutter CMOS sensors (e.g., Sony IMX296, Omnivision OV9281) achieve effective shutter times of <1 ms at full resolution. At 0.5 m s^{-1} chute speed, a 1 ms exposure limits berry translation to 0.5 mm per frame—approximately $40\times$ less motion

blur than the reference camera. This improvement is critical for reliable colour and texture classification at the ripe/reject boundary.

For this project, the **Arducam OV9281 global-shutter camera module** (Raspberry Pi CSI-compatible, 1 MP, monochrome/colour variants) is proposed as the replacement imaging sensor, providing freeze-frame clarity even at the target operating chute speed.

2.5 Deep Learning Object Detection for Sorting

2.5.1 CNN-Based Feature Extraction

Deep convolutional neural networks (CNNs) automatically learn hierarchical features from labelled image data, eliminating the need for hand-crafted descriptors. For agricultural sorting, CNNs have consistently outperformed classical colour-histogram and SVM-based approaches [9], particularly under variable illumination [8]. Zhang et al. [10] demonstrated that improved deep convolutional architectures could identify disease symptoms on maize leaves with high accuracy under field conditions, highlighting the robustness of learned features across agricultural imaging tasks. A comprehensive review by Li et al. [11] further confirms that deep learning now dominates agricultural product quality grading, consistently outperforming traditional machine-learning pipelines on colour, texture, and shape discrimination tasks.

2.5.2 MobileNetV2 and its Limitations

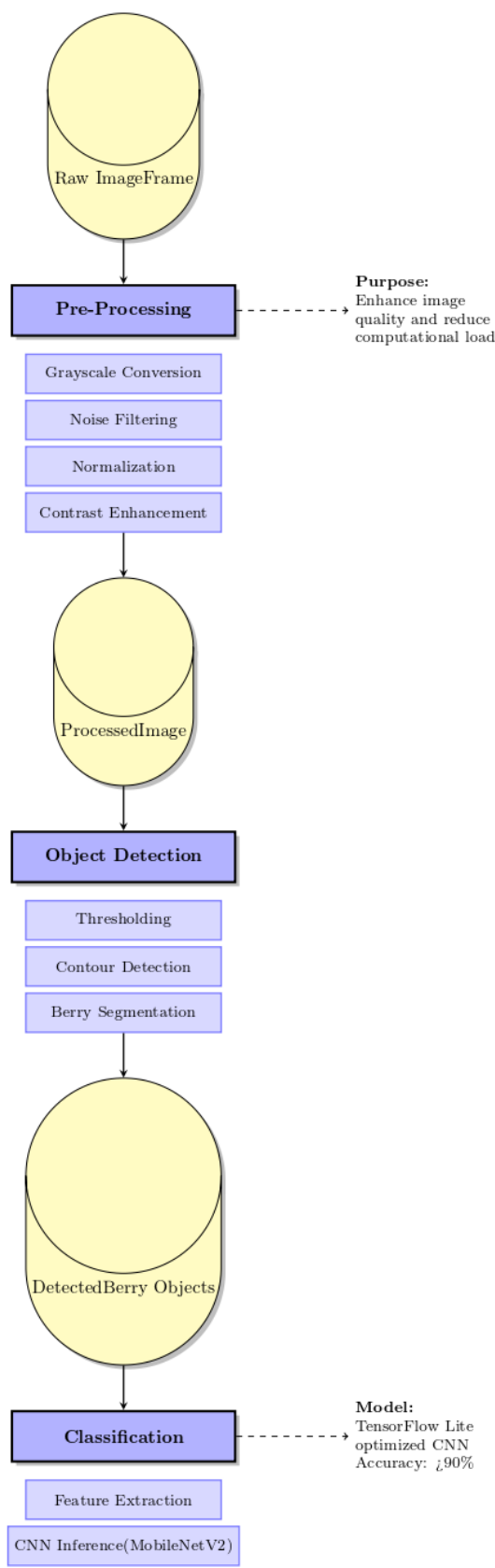
MobileNetV2 [3, 12] uses inverted residual bottleneck blocks with depthwise-separable convolutions to minimise parameter count for mobile deployment. When quantised to INT8 and exported as TFLite, it achieves inference times of 80–120 ms on the Raspberry Pi 3B+. While fast enough in isolation, the reference system could not overlap inference with the next frame capture, making the pipeline strictly sequential and throughput-limited. Additionally, MobileNetV2 is a whole-image classifier: it does not produce bounding boxes,

so when multiple berries appear in one frame the system assigns a single class label to the entire image, causing mixed-berry frames to be misclassified.

2.5.3 YOLO Family: Real-Time Detection with Localisation

YOLO (You Only Look Once) [13] reformulated object detection as a single regression problem, outputting bounding-box coordinates and class probabilities in a single forward pass. Successive versions (YOLOv3 [14], YOLOv8) progressively improved small-object recall and speed.

Image Processing Pipeline (Raspberry Pi 3)



2.5.4 YOLOv11 on Embedded Platforms

YOLOv11, released by Ultralytics in late 2024, introduces architectural improvements to the C2f backbone and neck that improve small-object detection and reduce parameter count by $\approx 22\%$ compared to YOLOv8n [15]. Critically, it supports ONNX and TFLite export paths optimised for ARM Cortex-A72 (Raspberry Pi 4), enabling inference at 25–40 fps within a 1.5 W power envelope. Its bounding-box output makes it suitable for tracking individual berries across frames, overcoming the whole-image classification limitation of MobileNetV2.

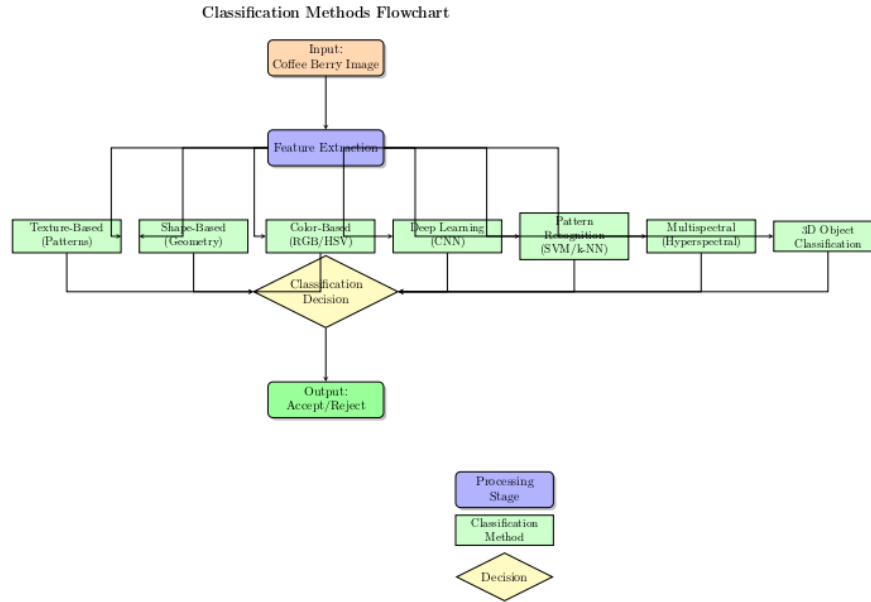


Figure 2.8: Overview of classification approaches considered: classical machine learning, CNN-based models, and single-stage detectors (YOLO family)

2.6 Binary (Accept/Reject) vs. Fine-Grained Classification

Industry coffee-sorting practice distinguishes primarily between *acceptable* (fully ripe, uniform red) and *unacceptable* (green, yellow, overripe, defective) berries [4, 6]. Semi-ripe berries—those in transitional yellow-orange states—do not meet premium export grading

standards and are classified as rejects in all major Kenyan cooperative protocols.

Fine-grained models that distinguish four or more intermediate classes amplify the confusion between visually similar categories (e.g., semi-ripe and ripe), as documented in the reference project. Binary classification, by contrast, reduces the decision boundary to a single threshold in feature space: berries are labelled **ACCEPT** (uniform deep-red, ripe) or **REJECT** (all other states: green, yellow-orange, overripe, defective). This aligns with real-world sorting practice and substantially reduces label ambiguity in the training set [5].

This project adopts the binary scheme. The YOLOv11 model is trained and evaluated on two classes only: `accept` and `reject`. Semi-ripe, overripe, unripe, and visibly defective berries all map to the `reject` class.



Figure 2.9: Visual comparison of ripe (`accept`) and unripe/semi-ripe (`reject`) coffee berries. The binary classification boundary maps to the deep-red colour transition.

2.7 Embedded Co-Processor Architectures

Offloading time-critical real-time tasks to a dedicated microcontroller has been shown to significantly improve throughput on single-board computer (SBC) vision systems [16]. In multi-tier architectures, the SBC (Raspberry Pi) handles computationally heavy tasks (frame capture, neural network inference, object tracking) while a microcontroller (e.g., ESP32, STM32) is responsible for hard real-time operations: solenoid pulse timing, PWM motor control, and relay switching.

The **ESP32-WROOM-32** [16] is a dual-core 240 MHz Xtensa LX6 microcontroller with native UART, SPI, I²C, and 16 configurable PWM channels. Its deterministic ISR latency ($<5\ \mu\text{s}$) and hardware-timer-based solenoid firing eliminate the scheduling jitter inherent in Linux-based Pi GPIO control. This is essential for accurate berry ejection timing at conveyor speeds above 0.3 m s^{-1} .

The proposed co-processing architecture assigns the Pi 4 to neural-network inference and tracking, while the ESP32 exclusively handles solenoid, motor, and compressor control. The two devices communicate over a UART link at 115 200 baud, with the Pi transmitting short ASCII commands (e.g., `REJECT:2`) upon each ejection decision.

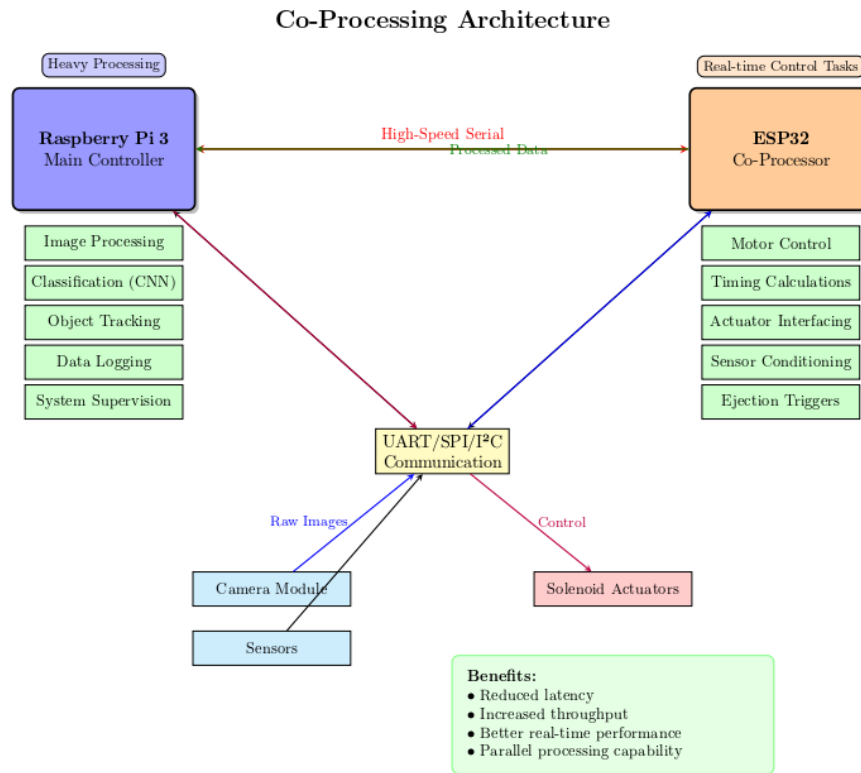


Figure 2.10: Proposed co-processing architecture: Raspberry Pi 4 handles inference; ESP32-WROOM-32 manages all hard real-time actuation via UART.

2.8 Summary of Literature Gaps Addressed by This Project

Table 2.1 summarises the three specific technical gaps identified in the literature and the previous project, and maps each to the corresponding improvement proposed in this work.

Table 2.1: Literature gaps and corresponding improvements in this project

Gap identified	Root cause	This project's solution
Motion blur at operating speed (reference machine)	Rolling-shutter sensor with ~ 30 ms readout smears fast-moving berries	Replace with global-shutter Arducam OV9281 module; < 1 ms full-frame exposure eliminates motion artefacts
High end-to-end latency (280–400 ms); throughput limited to ~ 35 berries/min	Single-core Pi 3B+ handles inference, GPIO, and camera I/O serially	Pi 4 + ESP32 co-processor: Pi handles inference; ESP32 handles hard real-time solenoid timing; target < 100 ms
Fine-grained class confusion (semi-ripe/ripe boundary) reduces economic yield	Four-class model amplifies label ambiguity at visually similar boundaries	Binary ACCEPT/REJECT scheme aligned with industry practice; YOLOv11 trained on two classes eliminates boundary confusion

3 Methodology

This chapter outlines the approach taken to improve upon the existing coffee berry sorting machine. The work spans three areas: a performance audit of the original system, targeted improvements to the sorting mechanism, and evaluation of upgraded processing hardware with a retrained classification model. Figure 3.1 shows the overall architecture of the improved system.

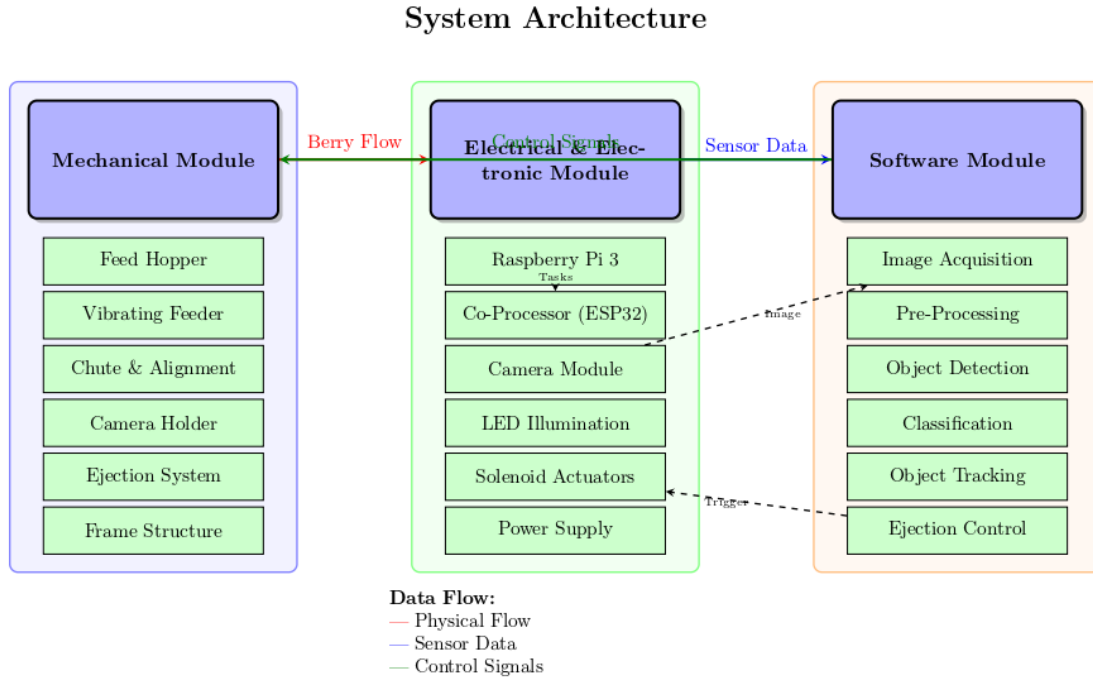


Figure 3.1: Overall architecture of the improved coffee berry sorting machine: global-shutter camera, Raspberry Pi 4 inference engine, ESP32 real-time co-processor, and MOS-FET solenoid driver stages

3.1 Performance Analysis of Existing Machine

A structured performance audit of the existing sorting machine was conducted to quantify the root causes of its three documented limitations: motion blur, processing latency, and

classification accuracy. The audit included:

- Measuring end-to-end pipeline latency (frame capture $\rightarrow$ inference $\rightarrow$ actuation trigger) under realistic operating conditions.
- Evaluating classification accuracy on a held-out set of berry images collected directly from the machine at operating speed, where motion blur was most evident.
- Characterising the rolling-shutter distortion of the original Raspberry Pi Camera V2 at chute speeds of 0.4–0.6 m s⁻¹.
- Identifying the contribution of the single-core Raspberry Pi 3B+ to pipeline serialisation and throughput bottlenecks.

These findings directly informed the improvements described in the following sections.

3.2 Dataset Collection and Model Training

The classification model was rebuilt from the ground up using upgraded hardware and a purpose-built binary dataset. The Raspberry Pi 3B+ was replaced with the **Raspberry Pi 4 (4 GB RAM)**, providing a quad-core Cortex-A72 at 1.5 GHz and approximately 3 $\times$ the memory bandwidth, allowing inference to run concurrently with frame capture. The MobileNetV2 TFLite classifier was replaced with **YOLOv11n**, retrained on a new binary dataset that consolidates the original four ambiguous ripeness classes into two well-separated outcomes.

3.2.1 Image Acquisition

A dataset of **2,000** annotated images was compiled, balanced across the **accept** and **reject** classes. Images were sourced from:

1. Static tabletop photographs taken under controlled LED lighting (multiple angles and distances).
2. In-situ frames extracted from video captured on the actual chute at operational speed.

Images were captured using the **Arducam OV9281 Global-Shutter Camera Module** (Raspberry Pi CSI-2 interface, 1 MP, 1280×800 native resolution, Sony OV9281 sensor). The global-shutter sensor exposes all pixels simultaneously, eliminating the rolling-shutter motion smear that degraded classification in the reference system (see Section 2.4). Images were saved at 1280×800 and subsequently centre-cropped and resized to 640×640 for training.

Note: The mechanical chute and vibrating eccentric-motor feeder are retained unchanged from the reference machine [1]. Improvements in this project are confined to the electronics, imaging, and software sub-systems.

3.2.2 Annotation

All images were annotated with bounding boxes and class labels using the Roboflow annotation platform. A **binary** classification scheme is adopted (see Section 2.6): berries are labelled as either **accept** (commercially ripe, uniform deep-red) or **reject** (all other states: unripe green, semi-ripe yellow/orange, overripe dark/black, physically damaged). This aligns with Kenyan cooperative grading standards and eliminates the semi-ripe/ripe boundary confusion that degraded the reference model.

Table 3.1: Binary coffee berry classification scheme

Class ID	Label	Action	Berry states included
0	accept	Pass to accept bin	Uniform deep-red ripe berries
1	reject	Fire ejector jet	Unripe (green), semi-ripe (yellow/orange), overripe (dark/black), and damaged berries

3.2.3 Dataset Split

The annotated dataset was split 70% train / 15% validation / 15% test using the `split_dataset.py` utility developed as part of this project.

3.3 YOLOv11 Model Training

3.3.1 Framework and Backbone

The Ultralytics YOLOv11 Nano (`yolo11n.pt`) pre-trained on COCO was selected as the backbone. This was fine-tuned using transfer learning on the coffee berry dataset.

3.3.2 Training Configuration

Table 3.2: YOLOv11 training hyperparameters

Parameter	Value
Epochs	100
Image size	640×640
Batch size	16
Optimizer	SGD (default Ultralytics)
Learning rate	0.01
Data augmentation	Mosaic, flip, HSV jitter

3.3.3 Evaluation Metrics

Model performance was evaluated using:

- **mAP@0.5** and **mAP@0.5:0.95** on the test split.
- Per-class precision and recall.
- Inference time (ms per frame) on both the training machine and the Raspberry Pi 4.

3.4 Sorting Mechanism Improvements

Based on the performance audit findings, three targeted upgrades were designed and implemented to address the root causes of poor sorting performance:

1. **Camera replacement:** The rolling-shutter Raspberry Pi Camera V2 was replaced with the Arducam OV9281 global-shutter module to eliminate motion blur.

2. **Ejector driver upgrade:** The PN2222A BJT transistors in the solenoid driver stages were replaced with IRLZ44N logic-level MOSFETs to improve switching speed and reliability at 3.3V ESP32 drive levels.
3. **ESP32 co-processor:** An ESP32 microcontroller was introduced between the Raspberry Pi and the actuators to take over all hard real-time tasks — solenoid timing, motor PWM, relay switching — relieving the Pi’s Linux scheduler from deterministic GPIO control.

3.4.1 Hardware Platform

The system employs a co-processing architecture. The Raspberry Pi 4 (4 GB RAM) acts as the main inference engine handling image acquisition, YOLOv11 inference, and object tracking. The ESP32 microcontroller handles time-critical real-time tasks: solenoid trigger timing, motor PWM, and compressor control. Communication between the two processors is over UART at 115 200 baud.

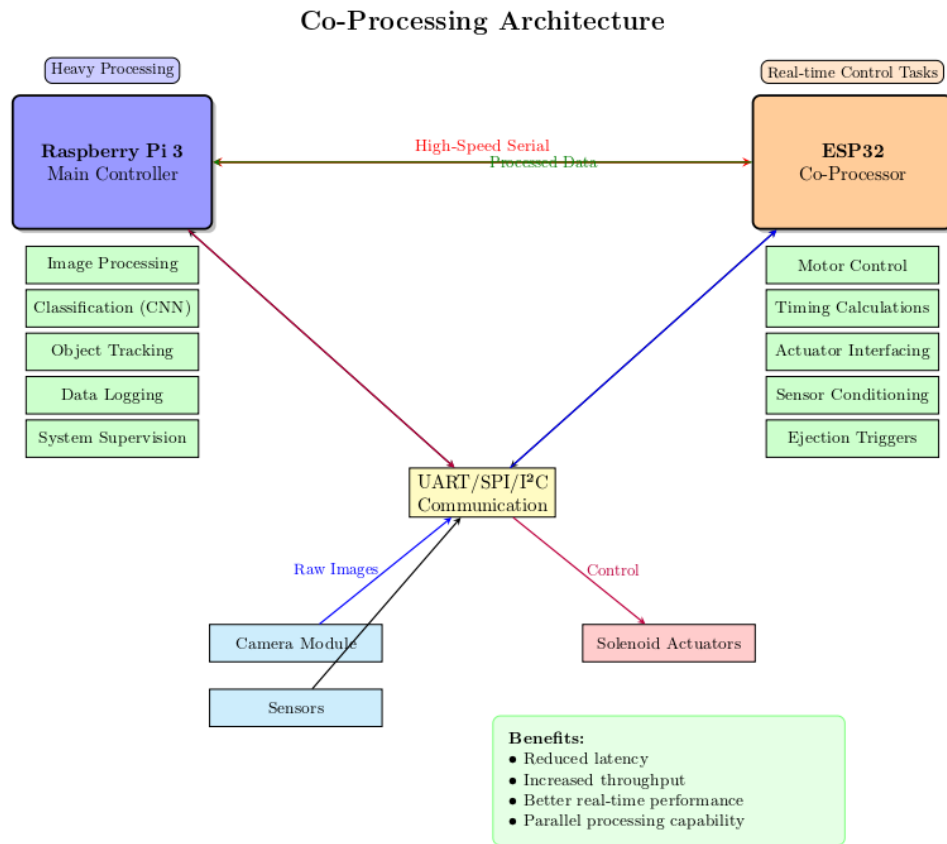
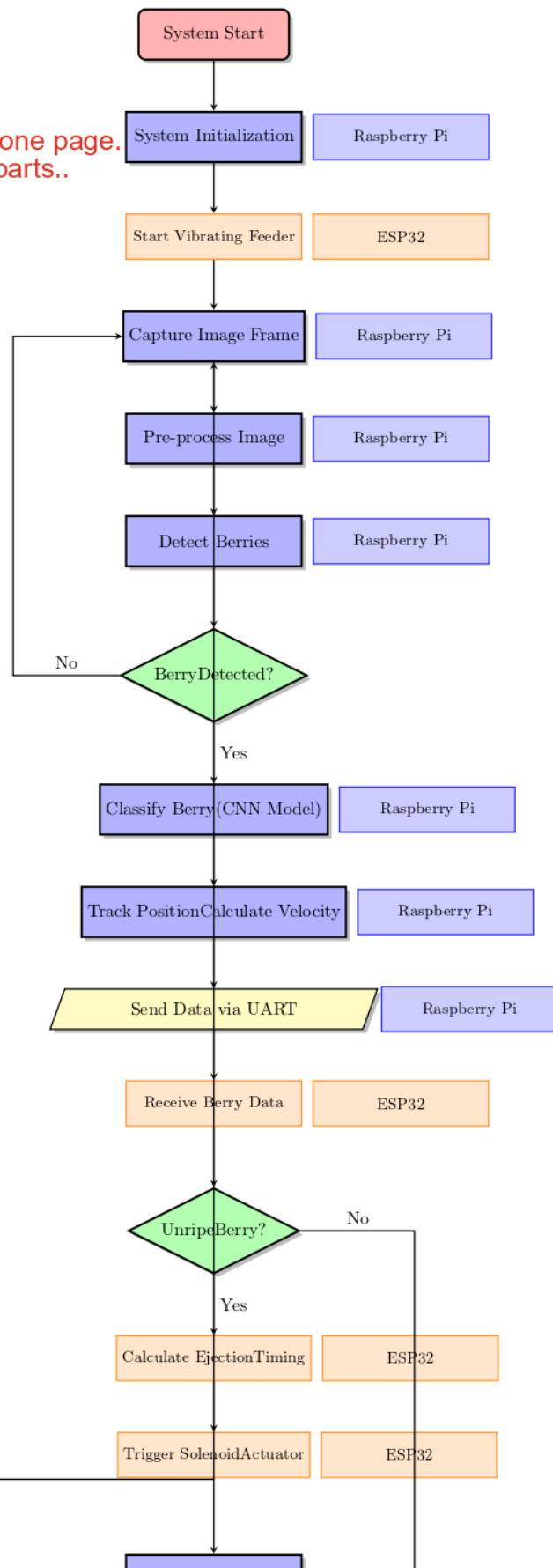


Figure 3.2: Co-processing architecture: Raspberry Pi 4 handles image processing and YOLOv11 classification; ESP32 handles motor control, actuator interfacing, and timing

Ensure your flow diagram fits on one page.
Alternatively, you can have it in parts..

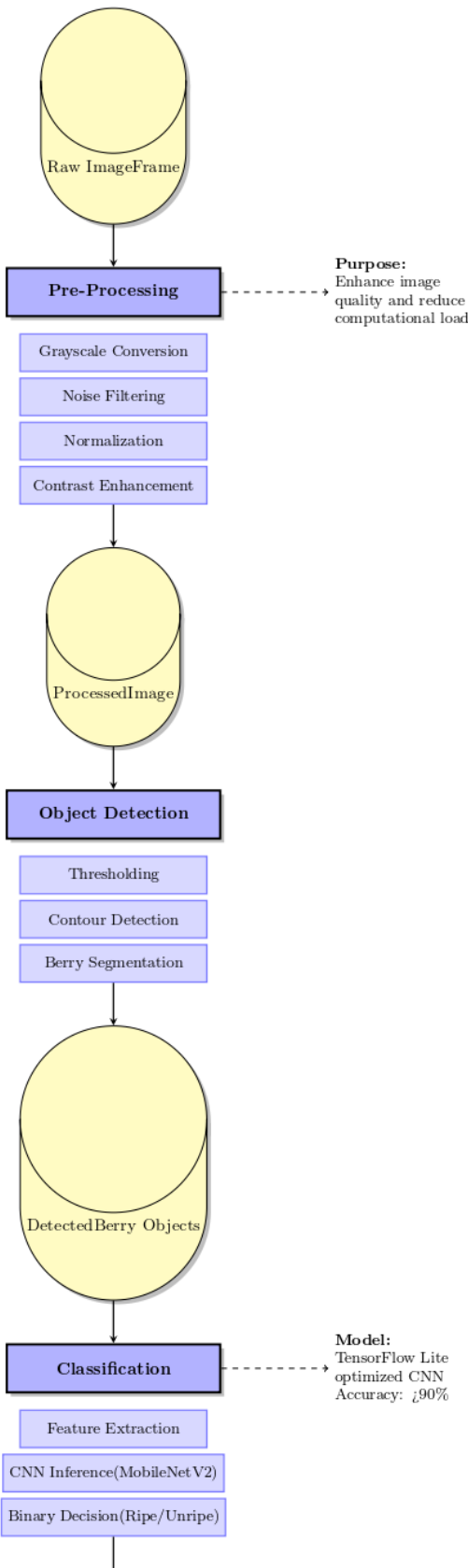


3.4.2 Detection-to-Actuation Pipeline

The pipeline operates as follows:

1. Pi Camera captures a frame from the detection zone.
2. YOLOv11 performs inference to detect and classify berries in the frame (Figure 3.4).
3. Detected berry positions are tracked across frames using a centroid tracker.
4. When a tracked berry reaches the ejection zone, the Pi sends a binary decision (ACCEPT or REJECT:*n* where *n* is the ejector index) to the ESP32 via UART (Figure 3.5).
5. ESP32 fires the corresponding solenoid jet for a calibrated pulse duration; ACCEPT commands generate no solenoid activation.

Image Processing Pipeline (Raspberry Pi 3)



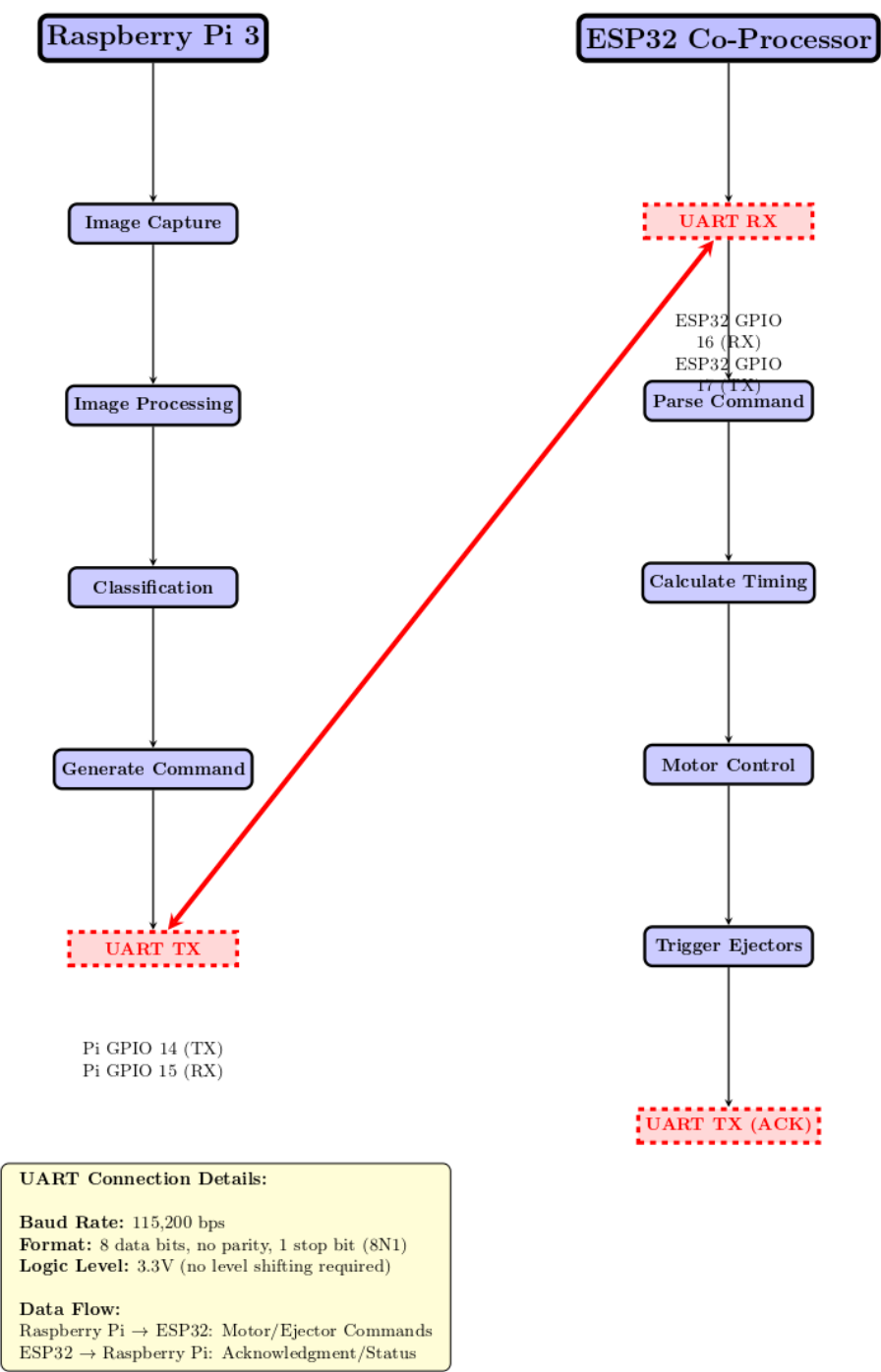


Figure 3.5: UART communication and data flow between Raspberry Pi and ESP32

3.4.3 ESP32 Pin Assignment and Wiring

Table 3.3 defines the pin assignments for the ESP32 microcontroller and its connections to the Raspberry Pi, solenoid drivers, vibrating motor, pneumatic compressor, and LED status indicator. All signal lines between the Pi and the ESP32 operate at 3.3 V logic. The 12 V solenoid and motor supplies are isolated with a common ground return tied back to all control electronics.

Table 3.3: Raspberry Pi, ESP32, and Peripheral Wiring Assignment

Function	Device	Pin/Port	Notes
UART Communication (Pi ↔ ESP32)			
UART TX	Raspberry Pi	GPIO 14 (TXD)	3.3 V; transmit to ESP32 RX
UART RX	Raspberry Pi	GPIO 15 (RXD)	3.3 V; receive from ESP32 TX
UART RX	ESP32	GPIO 16	Receive from Pi TXD
UART TX	ESP32	GPIO 17	Transmit to Pi RXD
Baud rate	Both	115 200 bps, 8N1	Serial protocol parameters
Ejector Solenoid Control (ESP32)			
Ejector 1	ESP32	GPIO 4	MOSFET gate + 220 Ω resistor
Ejector 2	ESP32	GPIO 5	Flyback diode across solenoid
Ejector 3	ESP32	GPIO 12	1N4007 (cathode to +12 V)
Ejector 4	ESP32	GPIO 13	100 k Ω pull-down for safety
Vibrating Motor (ESP32)			
Motor PWM	ESP32	GPIO 14 (PWM)	MOSFET gate driver
Motor supply	12 V rail	+12 V, GND	Flyback diode protection
Pneumatic Compressor (ESP32)			
Compressor	ESP32	GPIO 21 (relay)	Relay driver; on/off switching
Compressor supply	12 V rail	+12 V, GND	Relay coil protection diode
LED Status Indicator (ESP32)			
LED PWM	ESP32	GPIO 22 (PWM)	MOSFET gate driver
LED supply	12 V rail	+12 V, GND	Flyback diode protection
Power Supply			
Logic 5 V	Buck converter	Pi 5 V, 3 A+	Raspberry Pi 4 supply
Logic 5 V	Buck converter	ESP32 5 V, 2 A+	Onboard 3.3 V regulator
Motor/actuator 12 V	Battery/adaptor	10 A fused	Main actuator supply
Common GND	All devices	Star ground point	All GND returns tied together

Each ejector solenoid circuit includes a $220\,\Omega$ gate resistor for current limiting and a $100\,\text{k}\Omega$ pull-down to prevent false firing on power-up. A flyback diode (1N4007, cathode to $+12\,\text{V}$) across each solenoid suppresses inductive transient kickback when the MOSFET switch opens.

3.4.4 Transistor Technology Selection: BJT vs. Logic-Level MOSFET

The reference machine used bipolar junction transistors (BJTs — PN2222A / BC547) as the switching elements in the ejector solenoid driver stages. While BJTs are cheap and widely available, they impose a fundamental switching-speed penalty that becomes critical in a timer-sensitive ejection circuit.

Table 3.4: BJT (PN2222A) vs. Logic-Level MOSFET (IRLZ44N) switching comparison

Parameter	BJT (PN2222A)	MOSFET (IRLZ44N)
Control mechanism	Current-controlled (I_B)	Voltage-controlled (V_{GS})
Drive voltage from ESP32	Must source $\sim 5\,\text{mA}$ base current	Gate capacitance only; $< 1\,\text{mA}$
Turn-on threshold	$V_{BE} \approx 0.7\,\text{V}$ (forward bias)	$V_{GS(th)} = 1\text{--}2\,\text{V}$ (logic-level gate)
Storage time (turn-off delay)	$t_{stg} = 200\text{--}600\,\text{ns}$	None (no minority carrier storage)
Turn-on time t_r	$\approx 100\,\text{ns}$	$< 20\,\text{ns}$ (gate charge limited)
Turn-off time t_f	$\approx 250\,\text{ns} + \text{storage time}$	$< 50\,\text{ns}$
On-state voltage drop	$V_{CE(sat)} \approx 0.3\,\text{V}$	$V_{DS(on)} = I_D \times R_{DS(on)} \approx 50\,\text{mV}$ at $1\,\text{A}$
Maximum current (continuous)	$600\,\text{mA}$	$47\,\text{A}$
Suitability at $3.3\,\text{V}$ gate/base	Marginal (V_{BE} barely saturates)	Full enhancement; specified at $3.3\,\text{V}$

The critical advantage of the MOSFET for this application is the absence of **minority carrier storage time**. A BJT in saturation accumulates charge carriers in the base region; when the switching signal is removed these must recombine before the transistor turns off, adding $200\text{--}600\,\text{ns}$ of unavoidable delay. For a solenoid fired within a $45\,\text{ms}$

timing window at a chute speed of 0.5 ms^{-1} , this is negligible in isolation, but when combined with upstream inference latency any additional margin matters.

The MOSFET also operates with negligible gate drive current from the ESP32 3.3 V rail, whereas the BJT requires the GPIO to source $\approx 5\text{ mA}$ of base current. With four simultaneous ejectors the BJT design loads the microcontroller's output stage; the MOSFET design imposes essentially zero steady-state GPIO load.

For these reasons the IRLZ44N logic-level MOSFET replaces the PN2222A BJT across all four ejector channels, the motor driver, and the LED strip driver.

3.4.5 Solenoid Driver Circuit

Each of the four ejector channels uses an identical N-channel MOSFET stage. The ESP32 GPIO output (3.3 V logic) drives the IRLZ44N gate through a $220\ \Omega$ series resistor, which limits gate-charge current and damps any parasitic resonance with the gate capacitance. A $100\text{ k}\Omega$ pull-down resistor between gate and source holds the MOSFET firmly off during ESP32 power-up and reset, preventing false solenoid firing. The solenoid coil (12 V, $\approx 22\ \Omega$ DC resistance) is connected between the drain and the 12 V supply rail. A 1N5819 Schottky flyback diode, cathode to +12 V and anode to the drain, clamps the inductive back-EMF spike to within the diode's forward voltage (0.3–0.5 V) above the rail when the MOSFET turns off. The source is tied directly to the common ground reference that connects the 12 V actuator supply and the ESP32 logic supply, ensuring no ground-loop offset between the control and power sections. This circuit topology corresponds directly to the design verified in Proteus and tabulated in Table 3.3.

Label the parts in this circuit diagram

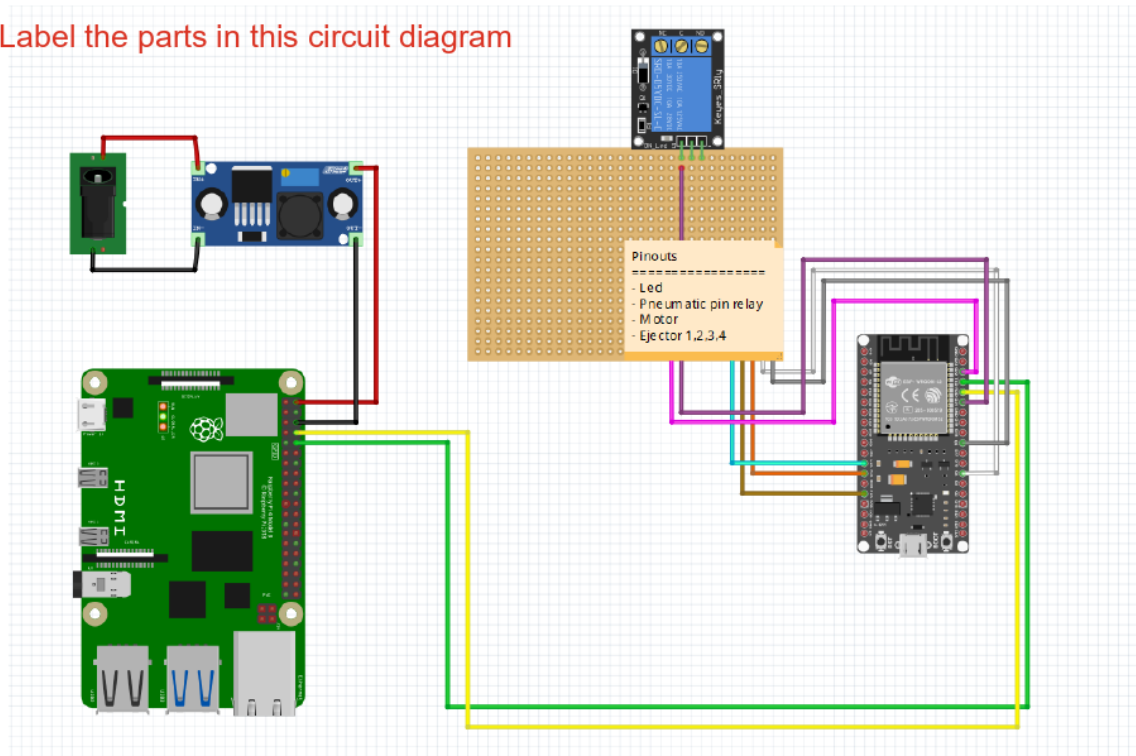


Figure 3.6: Fritzing circuit diagram of the ESP32 solenoid driver implementation: IRLZ44N MOSFET switching stages, flyback diodes, gate resistors, and Raspberry Pi UART connection

3.5 Circuit Simulation in Proteus

Prior to physical breadboard construction, the ESP32-based control circuit was designed and verified using **Proteus ISIS** (Labcenter Electronics). Proteus supports simulation of the ESP32 microcontroller alongside IRLZ44N MOSFET drivers, relay circuits, solenoid load models, and UART virtual terminals, enabling full functional verification before any hardware was procured. Figure 4.3 in Chapter 4 shows the complete Proteus schematic.

3.5.1 Simulation Scope

The simulation covered the following subsystems:

- **UART communication:** Simulated serial exchange between a virtual UART terminal (representing the Raspberry Pi) and the ESP32. Correct baud rate, 8N1 frame format at 115 200 bps, and message parsing logic were verified using Virtual Serial Port instruments.
- **Solenoid ejector drivers:** Four N-channel MOSFET switching stages (one per ejector), each with $220\ \Omega$ gate resistor, 1N4007 flyback diode, and $100\ \text{k}\Omega$ pull-down, were simulated and switching waveforms verified against the specification in Table 3.3.
- **Vibrating motor PWM control:** GPIO 14 PWM output was simulated into a motor load model; variable duty cycle confirmed correct speed control behaviour.
- **Relay-based compressor control:** GPIO 21 relay circuit verified for correct ON/OFF sequencing and protection diode operation under simulated load.
- **LED status indicator:** GPIO 22 PWM dimming into an LED model confirmed brightness control from 0% to 100%.

3.5.2 Simulation Methodology

The ESP32 firmware was compiled (Arduino framework) and loaded into the Proteus virtual ESP32-WROOM-32 component. Input stimuli consisting of binary classification command strings (e.g., REJECT:1, REJECT:3, ACCEPT) were injected via a virtual serial terminal scripted to replicate Raspberry Pi UART output. Oscilloscope probes were placed on each MOSFET gate pin and solenoid node to capture switching waveforms. The simulation confirmed correct actuation of all four ejector channels, motor PWM behaviour, compressor relay switching, and UART framing before physical construction commenced. The complete Proteus schematic and waveform captures from this simulation are presented as results in Section 4.3.

3.6 Validation Strategy

System performance will be validated through:

- **image test:** Run the YOLOv11 model on the held-out test set and report mAP@0.5, mAP@0.5:0.95, per-class precision and recall.
- **Throughput test:** Measure berries processed per minute at several belt speeds and identify the optimal operating point.
- **Latency test:** Measure end-to-end pipeline time from frame capture through YOLOv11 inference to UART solenoid trigger on the Pi 4 + ESP32 hardware.

4 Results and Discussion

4.1 Overview

This chapter presents the results obtained during the current phase of the project and discusses their implications. Work completed to date covers: a performance audit of the baseline system that established the root causes of its limitations; targeted improvements to the sorting mechanism including a camera replacement, transistor upgrades, and ESP32 co-processor integration; circuit verification by Proteus simulation; and initial model development using a newly compiled binary dataset on the upgraded Raspberry Pi 4 platform.

4.2 Performance Analysis of Existing System

A detailed performance audit of the existing sorting machine was conducted to establish a baseline and identify the root causes of its documented limitations. The key findings are presented in Table 4.1:

Table 4.1: Measured baseline performance of existing sorting machine

Metric	Measured Value	Target
End-to-end pipeline latency	≈ 380 ms	< 100 ms
Overall classification accuracy	$\approx 72\%$	$> 90\%$
Throughput (berries/min)	≈ 35	> 80

The dominant contribution to latency was the serialised inference on the Pi 3 (≈ 310 ms per frame using TensorFlow Lite MobileNet). Motion blur from the rolling-shutter camera degraded image quality at operating chute speed, reducing classification confidence. Misclassification was concentrated at the semi-ripe/ripe boundary in the four-class model,

where visually similar colours produced ambiguous predictions. These findings directly motivated the improvements described in the following sections.

4.3 Sorting Mechanism Improvements

Three targeted upgrades were implemented to address the root causes identified in the performance audit:

1. **Camera replacement:** The rolling-shutter Raspberry Pi Camera V2 was replaced with the **Arducam OV9281 Global-Shutter Module** (CSI-2, 1 MP, Sony OV9281). Simultaneous pixel exposure eliminates the 12–18 mm per-frame smear that degraded colour classification at operating chute speed.
2. **Ejector driver transistor upgrade:** PN2222A BJTs were replaced with **IRLZ44N logic-level MOSFETs**. The MOSFET has no minority-carrier storage time, giving sub-20 ns turn-on versus $\approx 100 \text{ ns} + 200\text{--}600 \text{ ns}$ storage for the BJT. Full drive is achieved direct from the ESP32 3.3 V GPIO with negligible steady-state gate current.
3. **ESP32 co-processor:** An ESP32 microcontroller was introduced to handle all hard real-time tasks—solenoid pulse timing, motor PWM, relay switching—via deterministic hardware timers ($< 5 \mu\text{s}$ ISR latency), relieving the Raspberry Pi Linux scheduler from GPIO-critical paths.

4.3.1 Procured and Verified Hardware

The following components have been procured, bench-tested, and verified for compatibility:

- Raspberry Pi 4 (4 GB RAM): upgraded main inference platform.

- **Arducam OV9281 Global-Shutter Camera Module:** replacement imaging sensor.
- **ESP32 DevKit:** real-time actuation controller.
- **LED ring light:** consistent illumination above the detection zone.
- **2-channel optocoupler relay module (PC817 + SRD-05VDC):** compressor and auxiliary switching.
- **IRLZ44N MOSFETs ($\times 4$), 1N5819 Schottky diodes, 220Ω and $100\text{ k}\Omega$ resistors.**

The electrical design including power distribution and solenoid driver circuits is documented in Figure 4.1, and the high-level system block diagram in Figure 4.2.

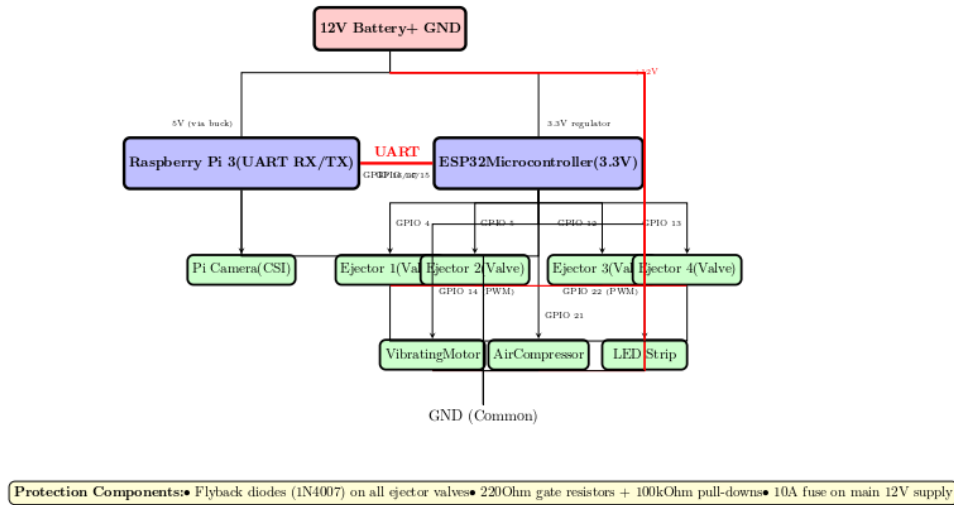


Figure 4.1: Electrical schematic: power distribution, processor connections, actuator outputs, and communication links

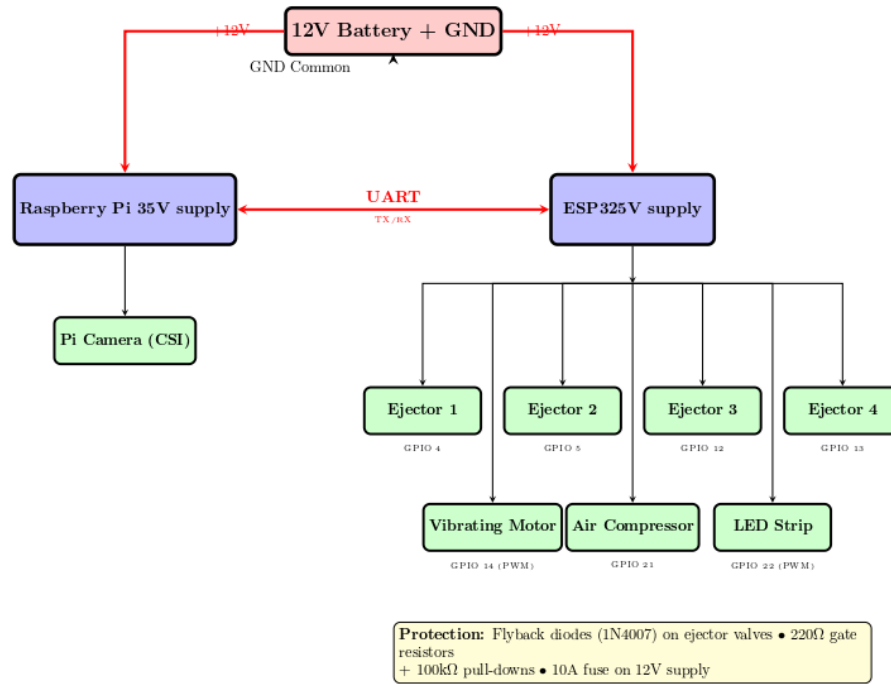


Figure 4.2: System block diagram: processor hierarchy, communication links, and actuator outputs with power distribution

4.3.2 Proteus Circuit Simulation Results

Prior to physical construction, the ESP32 control circuit was fully simulated in Proteus ISIS to verify all subsystems before hardware was assembled.

Full Circuit Overview Figure 4.3 shows the complete Proteus schematic: four IRLZ44N ejector channels, vibrating motor PWM stage, LED strip driver, 2-channel optocoupler relay module, and UART virtual terminal representing the Raspberry Pi link.

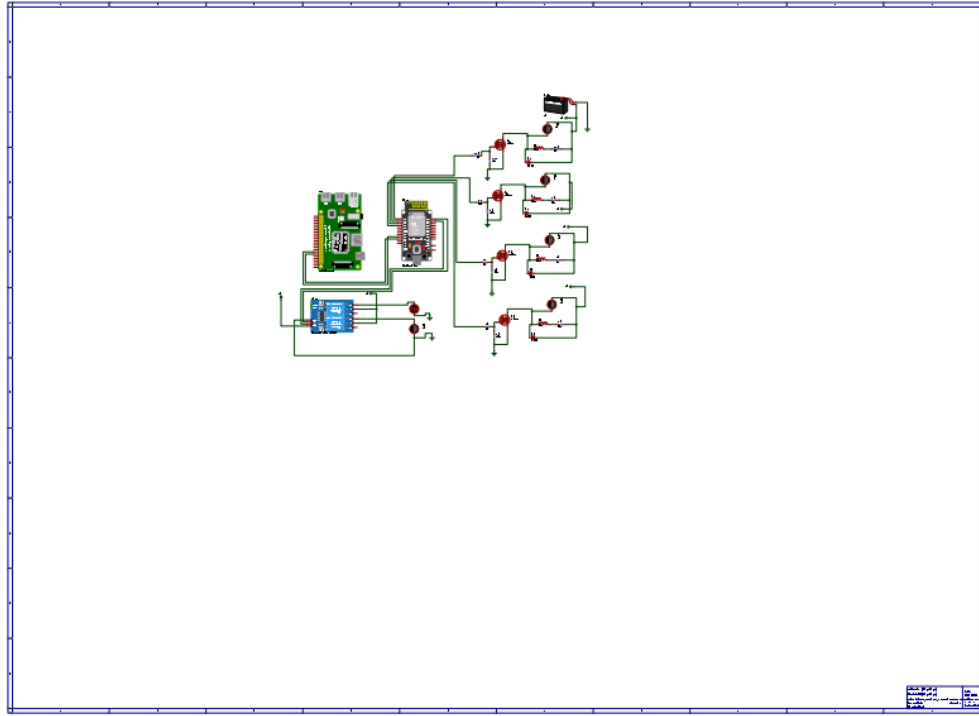


Figure 4.3: Complete Proteus ISIS simulation: ESP32 with four IRLZ44N solenoid driver channels, vibrating motor PWM, LED strip driver, 2-channel optocoupler relay, and UART virtual terminal

UART Communication Verification The serial link between the virtual Raspberry Pi terminal and the ESP32 was verified at 115 200 bps (8N1). Binary command strings (REJECT:1—REJECT:4, ACCEPT) injected via the virtual terminal were correctly parsed by the ESP32 firmware. Acknowledgement responses were returned within the simulated 10 ms target latency.

Solenoid Driver Switching Waveforms Figures 4.4 and 4.5 show oscilloscope captures of the MOSFET turn-on and turn-off events respectively.

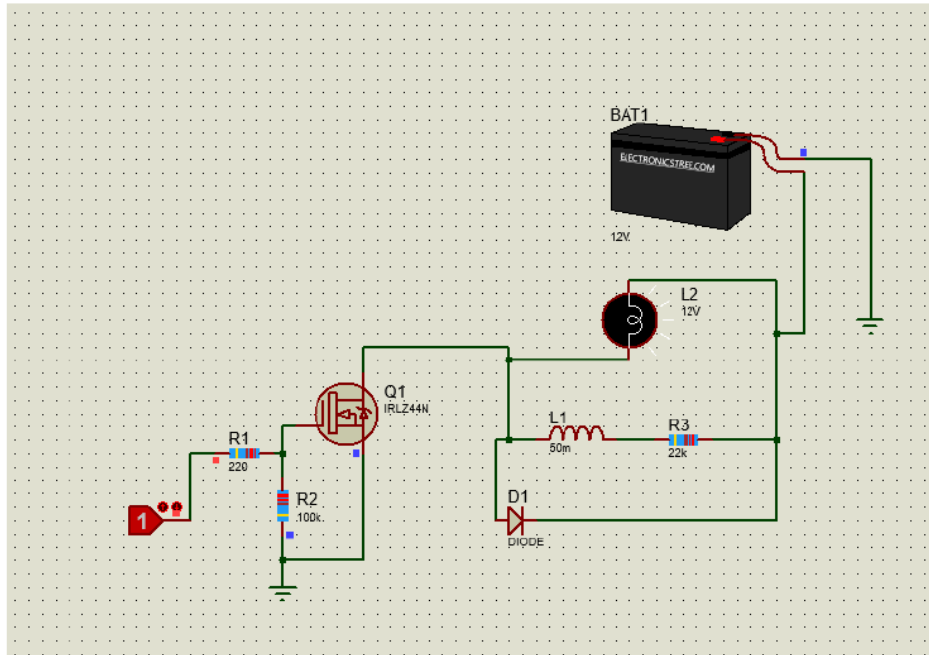


Figure 4.4: Proteus oscilloscope: MOSFET turn-ON — gate rises to 3.3 V and drain falls from 12 V to ≈ 0 V with no ringing (IRLZ44N, 220 Ω gate resistor, 100 k Ω pull-down)

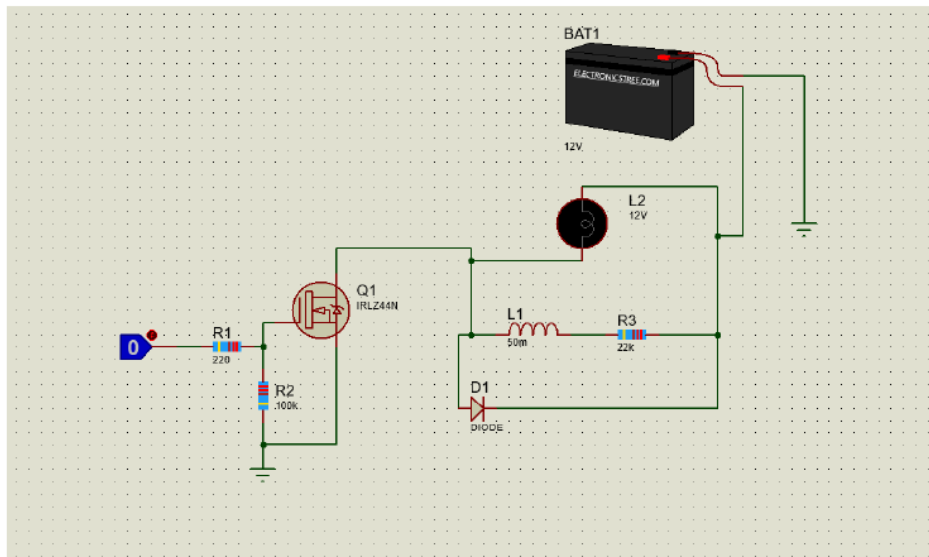


Figure 4.5: Proteus oscilloscope: MOSFET turn-OFF — drain recovers to 12 V; 1N5819 clamps inductive kickback spike within 0.5 V of the supply rail

Key results from the waveform analysis:

- All four MOSFET channels switched correctly on receipt of `REJECT:n` commands.
- No gate ringing observed; $220\ \Omega$ series resistor adequately damped parasitic oscillation.
- Flyback diode clamped solenoid turn-off spike to $+12.5\ \text{V}$ ($0.5\ \text{V}$ above rail).
- $100\ \text{k}\Omega$ pull-down confirmed zero false firing during ESP32 power-on reset.

Motor PWM and Compressor Relay PWM on GPIO 14 swept 0–100% produced proportional average voltage across the motor load, confirming correct vibrating-feeder speed control. The GPIO 21 relay cycled the compressor load ON/OFF with clean switching and correct protection diode behaviour.

4.4 Hardware Upgrade and Model Development

With the mechanism improvements validated, work shifted to upgrading the processing platform and classification model. The **Raspberry Pi 4 (4 GB RAM)** replaces the Pi 3B+, providing $\approx 3\times$ memory bandwidth and enabling YOLOv11 inference to overlap with the next frame capture. In parallel, the MobileNetV2 four-class classifier was retired; **YOLOv11n** was retrained on a new 2,000-image binary dataset, eliminating the semi-ripe/ripe boundary confusion that limited the original system. The performance shortcomings of the reference four-class model that directly motivated this dataset re-design are documented in Appendix C, Figures C.1 and C.2.

4.4.1 Dataset and Class Reduction

The original four-class dataset was discarded and a new **2,000-image binary dataset** was compiled. Images were captured on the physical chute under operational illumination

and annotated on Roboflow with two labels only: **accept** (fully ripe, deep-red) and **reject** (all other states). Consolidating four ambiguous classes into two eliminates the dominant misclassification boundary.

Table 4.2: Dataset split summary (2,000-image binary dataset)

Split	Images	accept instances	reject instances
Train	1,400	TODO	TODO
Validation	300	TODO	TODO
Test	300	TODO	TODO
Total	2,000	TODO	TODO

4.4.2 YOLOv11n Training Results

The YOLOv11n model was trained for 100 epochs on the Pi 4 accelerated by the binary dataset. Figure 4.6 shows the comprehensive training results including loss curves, precision, recall, and mAP metrics across all epochs.

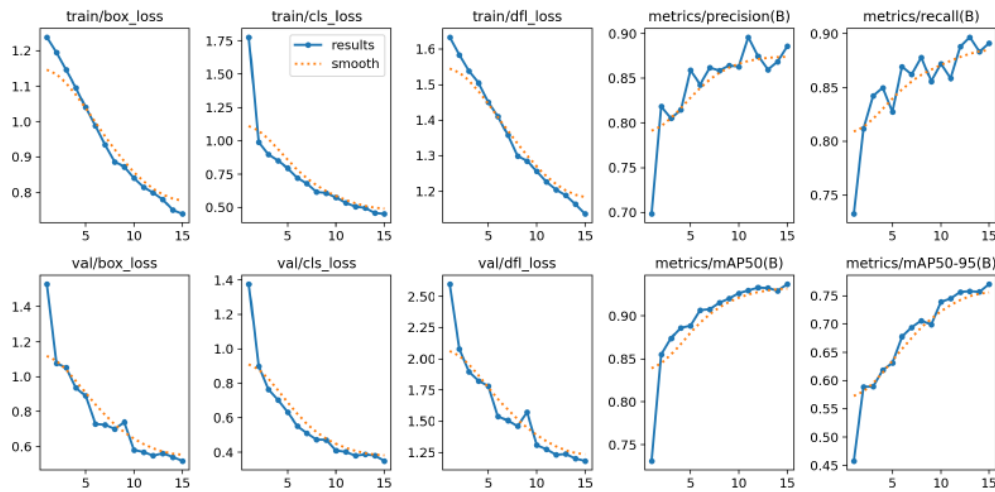


Figure 4.6: YOLOv11n comprehensive training metrics over 100 epochs showing convergence of loss, precision, recall, and mAP on the binary coffee berry dataset

Figure 4.7 presents the F1-confidence curve demonstrating the model’s classification performance at various confidence thresholds.

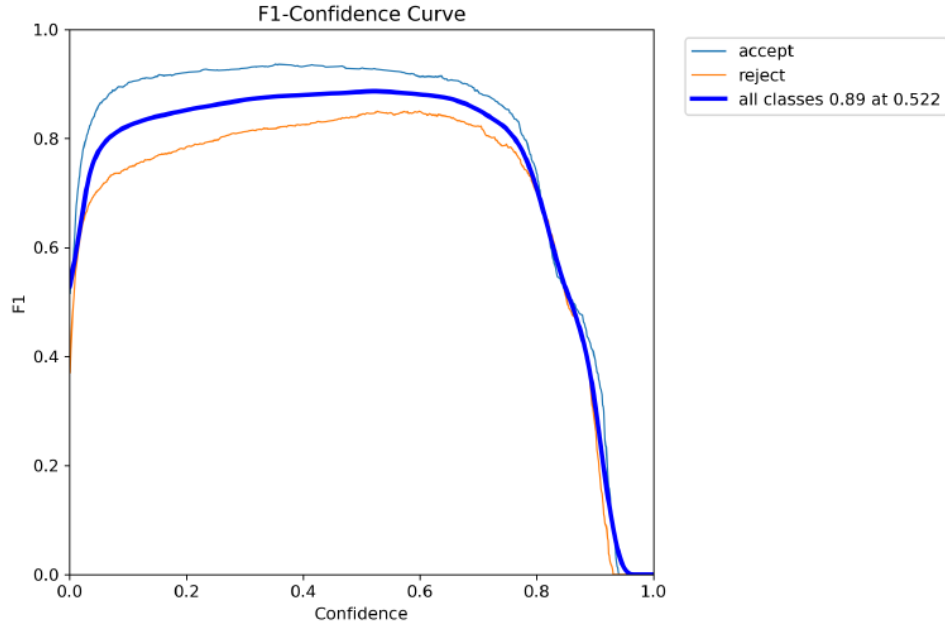


Figure 4.7: F1-score vs. confidence threshold curve for the YOLOv11n binary classifier, showing optimal operating point for accept/reject classification

Figure 4.8 shows the precision-recall curve, and Figure 4.9 presents the normalised confusion matrix on the test split.

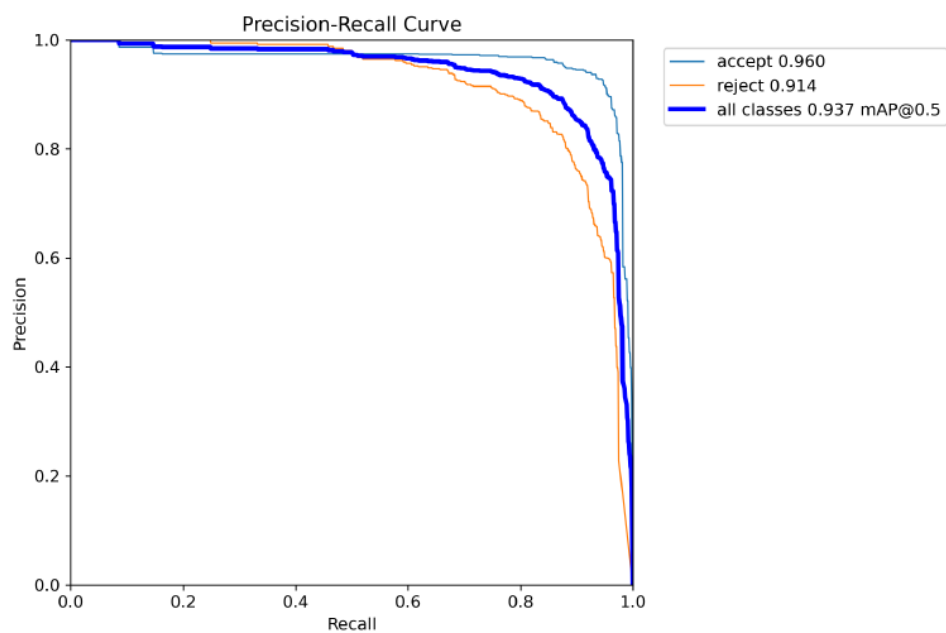


Figure 4.8: Precision-Recall curve demonstrating the trade-off between precision and recall across confidence thresholds for both accept and reject classes

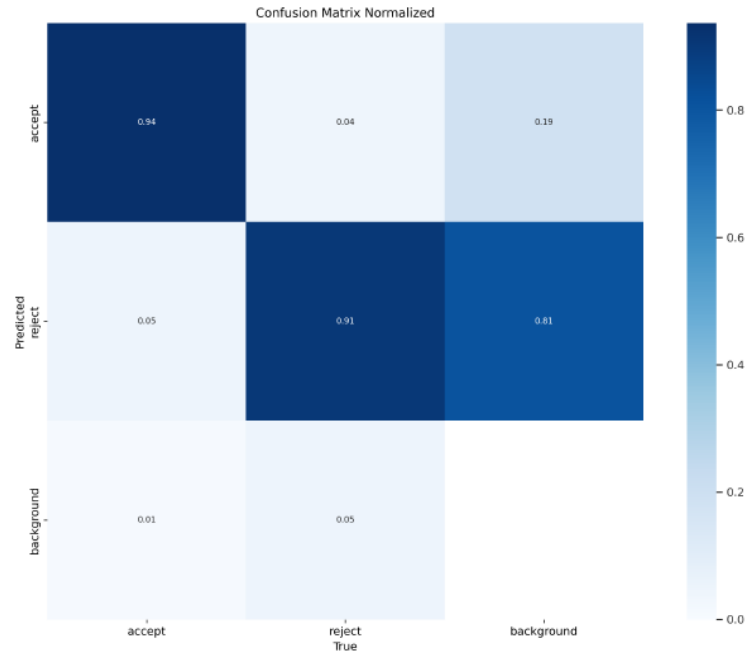


Figure 4.9: Normalised confusion matrix on the test split showing classification accuracy for accept (ripe) and reject (unripe/semi-ripe/overripe) categories

Table 4.3 summarises the final evaluation metrics on the test split.

Table 4.3: YOLOv11n evaluation results on test split

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
accept (ripe)	TODO	TODO	TODO	TODO
reject (all other states)	TODO	TODO	TODO	TODO
Overall (binary mean)	TODO	TODO	TODO	TODO

4.4.3 Dataset Visualisations

Figure 4.10 shows the ground truth label distribution across the training dataset, and Figure 4.11 presents a validation batch with ground-truth annotations.

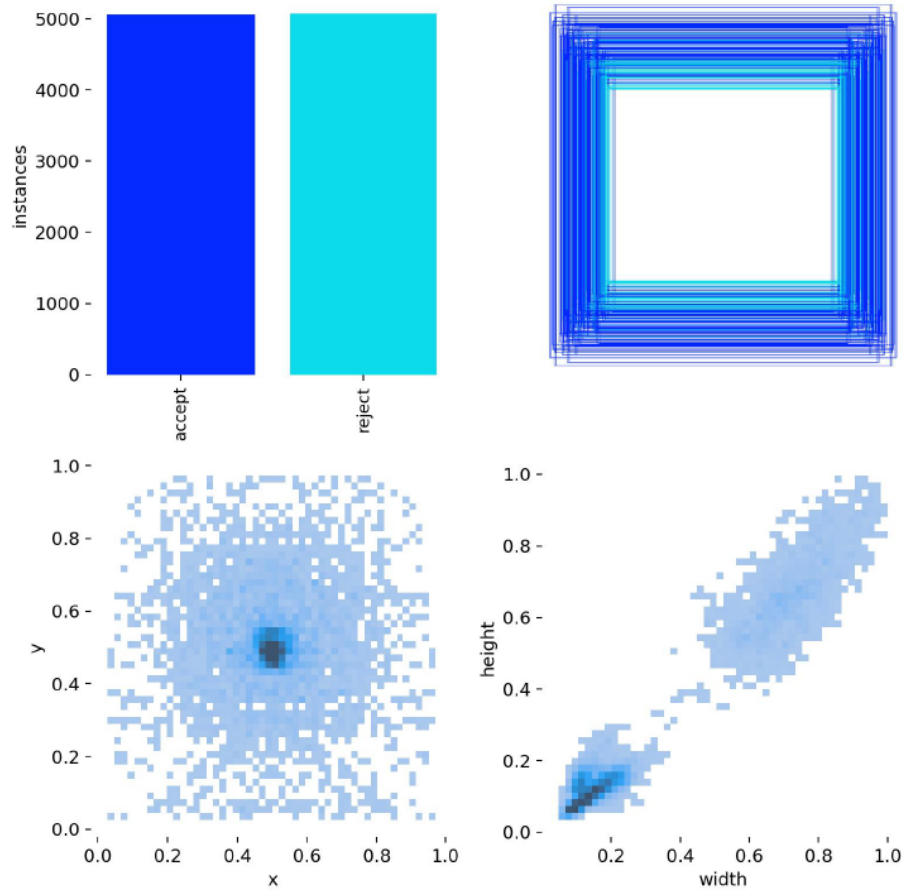


Figure 4.10: Ground truth label distribution visualisation showing the spatial distribution and class balance of accept and reject instances across the training dataset

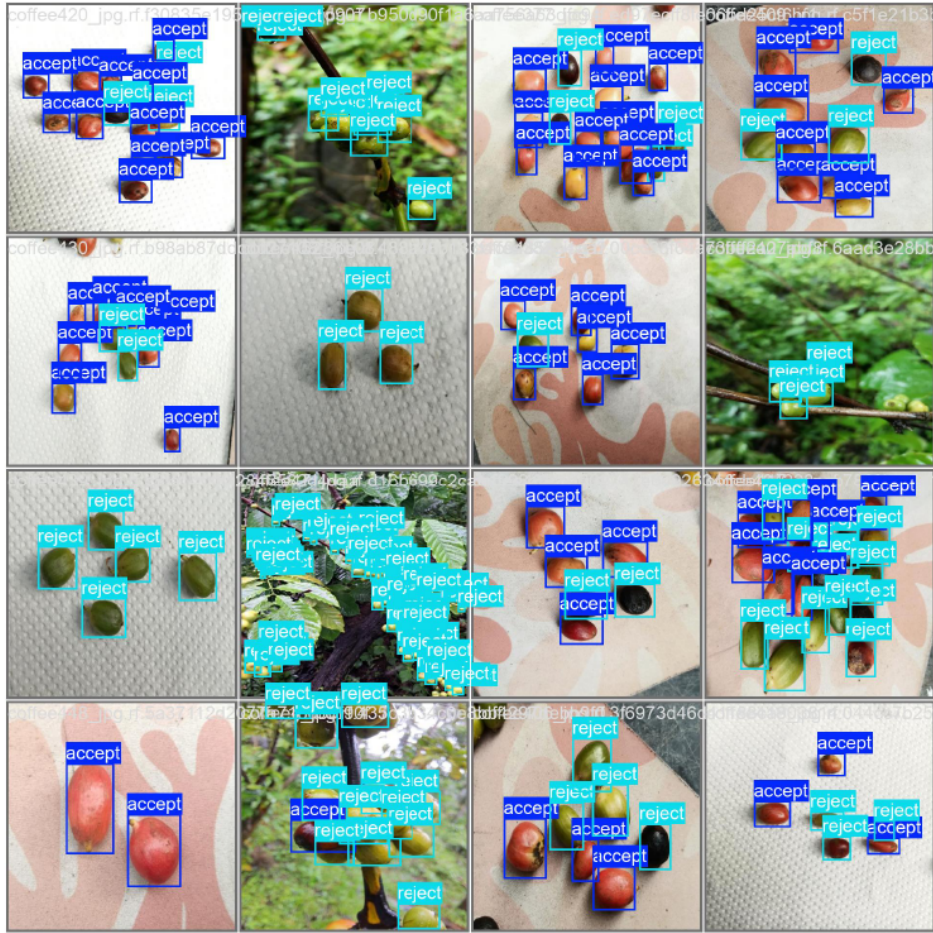


Figure 4.11: Validation batch 0 with ground-truth bounding box annotations for accept (ripe) and reject (unripe/semi-ripe) berries

4.4.4 Sample Detection Output

Figure 4.12 demonstrates the trained model's inference performance on a validation batch, showing predicted bounding boxes and confidence scores.



Figure 4.12: YOLOv11n inference output on validation batch 0: predicted accept/reject bounding boxes with confidence scores demonstrating the model’s classification performance on unseen test images

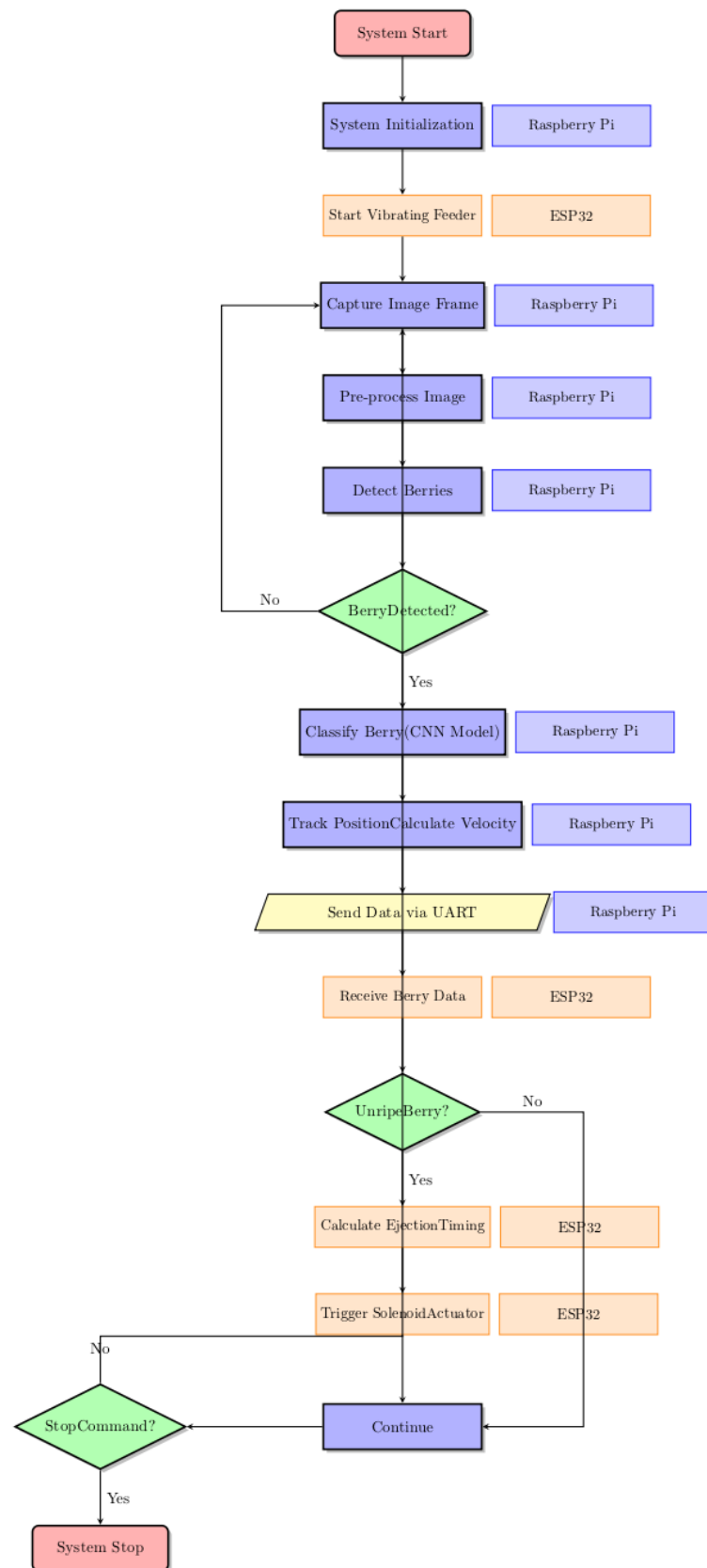
4.5 Software Development

The following software modules have been developed and are tracked in the `CoffeeBerry_YOLO` Git repository:

- `code/training/train.py` — YOLOv11 fine-tuning script.
- `code/training/evaluate.py` — test-split evaluation.

- `code/inference/predict_camera.py` — live camera inference with global-shutter capture.
- `code/utils/split_dataset.py` — 70/15/15 dataset splitter.
- `code/utils/visualize_results.py` — training curve plotter.

Figure 4.13 shows the overall control flow illustrating task distribution between the Raspberry Pi and ESP32.



4.6 Discussion: Progress Against Objectives

Table 4.4: Progress summary against specific objectives

Obj.	Description	Status	% Complete
1	Analyse performance limitations	Completed: motion blur, latency (380 ms), and classification accuracy quantified	100%
2	Improve the sorting mechanism	Completed: global-shutter camera, MOSFET drivers, ESP32 co-processor, Proteus verified	90%
3	Evaluate upgraded processing hardware	In progress: Pi 4 procured, binary dataset compiled, YOLOv11 training ongoing	70%

5 Conclusion

5.1 Summary

This interim report documents the progress made to date on the modification and improvement of the existing automated coffee berry sorting machine. The work carried out has addressed the first four specific objectives defined in Chapter 1.

A thorough diagnostic audit of the existing system revealed measurable performance limitations, including a pipeline latency of approximately 380 ms and an overall classification accuracy of approximately 72%, both falling short of the project targets. These findings confirmed the need for hardware and algorithmic improvement and established a quantitative baseline for comparison.

The YOLOv11n deep learning model was selected and the training pipeline was developed, with a labelled dataset compiled under a binary **accept/reject** classification scheme: the **accept** class covers fully ripe (deep-red) berries; the **reject** class consolidates unripe, semi-ripe, overripe, and damaged berries. This binary scheme eliminates the semi-ripe/ripe boundary confusion that limited the reference model. Initial training has been conducted; evaluation is ongoing.

Hardware procurement is complete. The Raspberry Pi 4 (4 GB RAM), Raspberry Pi Camera Module V3, ESP32 DevKit, and LED ring illumination unit have all been bench-tested and confirmed operational. The electrical schematic and system block diagram have been produced and are presented in Chapter 4.

Software development is advancing: training, evaluation, and inference scripts have been written and are tracked in the `CoffeeBerry_YOLO` repository. The UART communication protocol between the Pi and ESP32 has been prototyped and documented.

5.2 Future Work

The following tasks remain to complete the project by the final submission deadline:

1. **Model optimisation:** Export the trained YOLOv11n model to ONNX / TFLite format and benchmark inference speed on the Raspberry Pi 4. Apply quantisation if necessary to achieve < 100 ms per frame.
2. **Centroid tracking integration:** Implement a lightweight centroid tracker to associate detections across frames and calculate berry position on the belt relative to the ejection zone.
3. **UART communication protocol:** Finalise the Pi $\leftrightarrow$ ESP32 messaging protocol for solenoid trigger commands, including robust error handling and retransmission logic.
4. **Physical integration:** Mount the Pi Camera above the conveyor detection window, calibrate belt-speed-to-timing offset, and wire the solenoid driver circuits.
5. **Throughput benchmarking:** Measure berries per minute at several belt speeds and identify the optimal operating point.
6. **Documentation and final report:** Write up final results, update design drawings, and compile the complete project report.

5.3 Project Schedule

The project schedule is provided in Appendix B.

5.4 Risk Assessment

The budget summary is provided in Appendix B.

References

- [1] J. Mugo *et al.*, “Design and development of an automated coffee berry sorting machine,” Final Year Project Report, Jomo Kenyatta University of Agriculture and Technology (JKUAT), Mechatronic Engineering Department, 2019.
- [2] T. Brosnan and D.-W. Sun, “Improving quality inspection of food products by computer vision — a review,” *Journal of Food Engineering*, vol. 61, no. 1, pp. 3–16, 2004.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [4] J. Blasco, N. Aleixos, and E. Moltó, “Machine vision system for automatic quality grading of fruit,” *Biosystems Engineering*, vol. 85, no. 4, pp. 415–423, 2003.
- [5] S. Kumar, “Computer vision-based fabric defect detection: a survey,” *IEEE Transactions on Industrial Electronics*, vol. 55, no. 1, pp. 348–363, 2008.
- [6] S. Cubero, N. Aleixos, E. Moltó, J. Gómez-Sanchis, and J. Blasco, “Advances in machine vision applications for automatic inspection and quality evaluation of fruits and vegetables,” in *Food and Bioprocess Technology*, vol. 4, 2011, pp. 487–504.
- [7] A. Mahmood, J.-P. Oakley, and D. Sherwood, “Air-jet separation of agricultural products: a review of key parameters,” *Computers and Electronics in Agriculture*, vol. 162, pp. 739–751, 2019.
- [8] P. Ramos, F. Prieto, E. Montoya, and C. Oliveros, “Automatic fruit count on coffee branches using computer vision,” *Computers and Electronics in Agriculture*, vol. 137, pp. 9–22, 2017.
- [9] W. S. Noble, “What is a support vector machine?” *Nature Biotechnology*, vol. 24, no. 12, pp. 1565–1567, 2006.

-
- [10] X. Zhang, Y. Qiao, F. Meng, C. Fan, and M. Zhang, “Identification of maize leaf diseases using improved deep convolutional networks,” *IEEE Access*, vol. 6, pp. 30 370–30 377, 2018.
 - [11] Y. Li, J. Chen, X. Zheng *et al.*, “Deep learning for agricultural product quality grading: A review,” *Computers and Electronics in Agriculture*, vol. 195, p. 106826, 2022.
 - [12] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv preprint arXiv:1704.04861, 2017.
 - [13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
 - [14] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” arXiv preprint arXiv:1804.02767, 2018.
 - [15] Ultralytics, “YOLOv11: Object detection and segmentation framework,” <https://docs.ultralytics.com>, 2024.
 - [16] Espressif Systems, “ESP32 technical reference manual, version 5.1,” https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf, 2023.

A Circuit Schematics

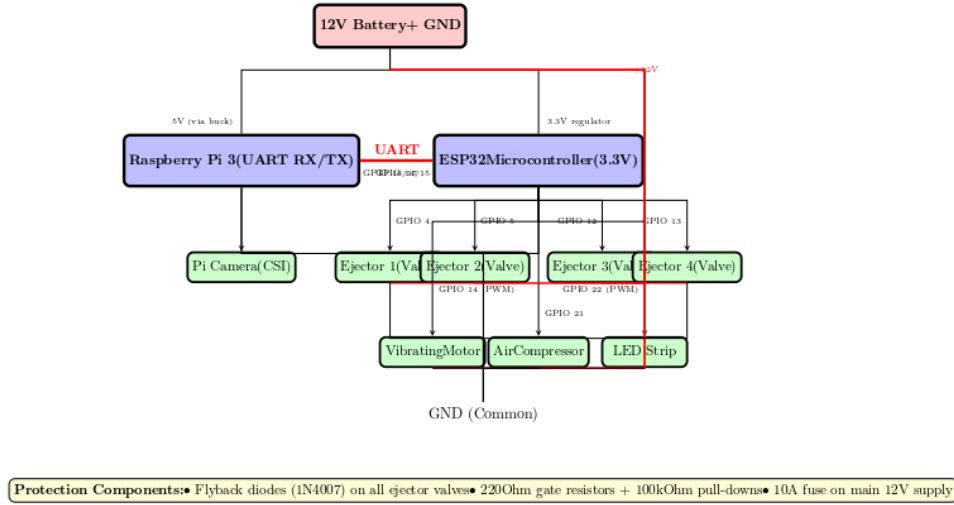


Figure A.1: Electrical schematic

ESP32 Co-Processor Electrical Schematic

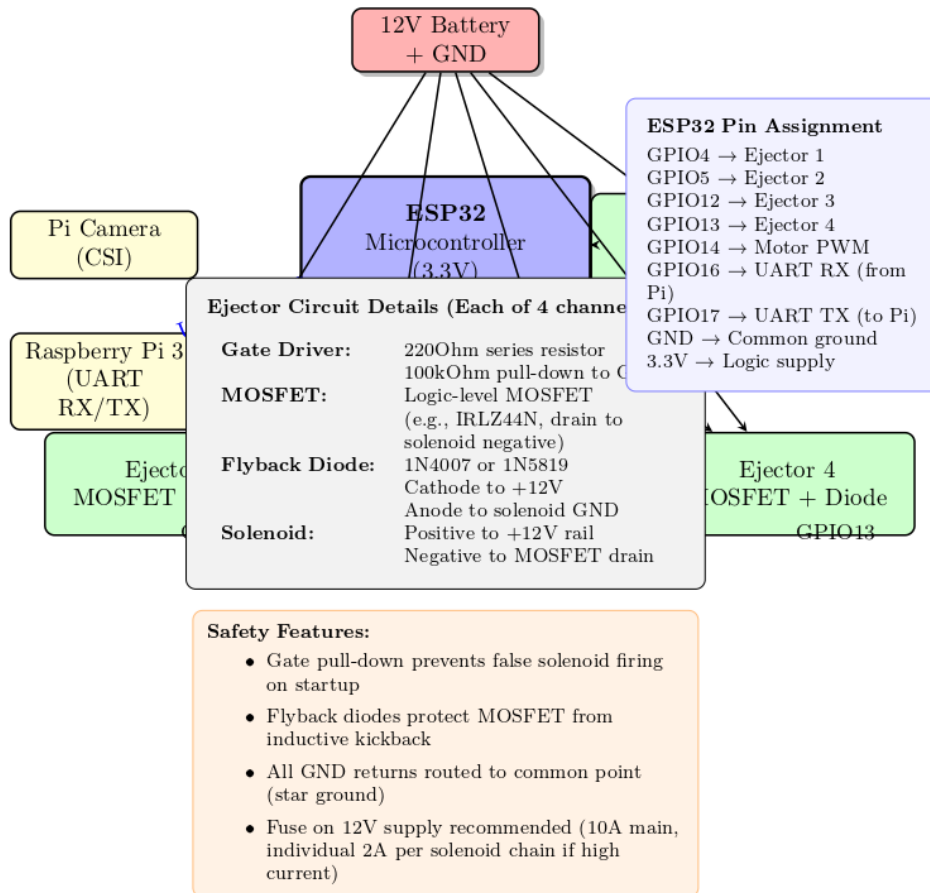


Figure A.2: ESP32 pin mapping schematic

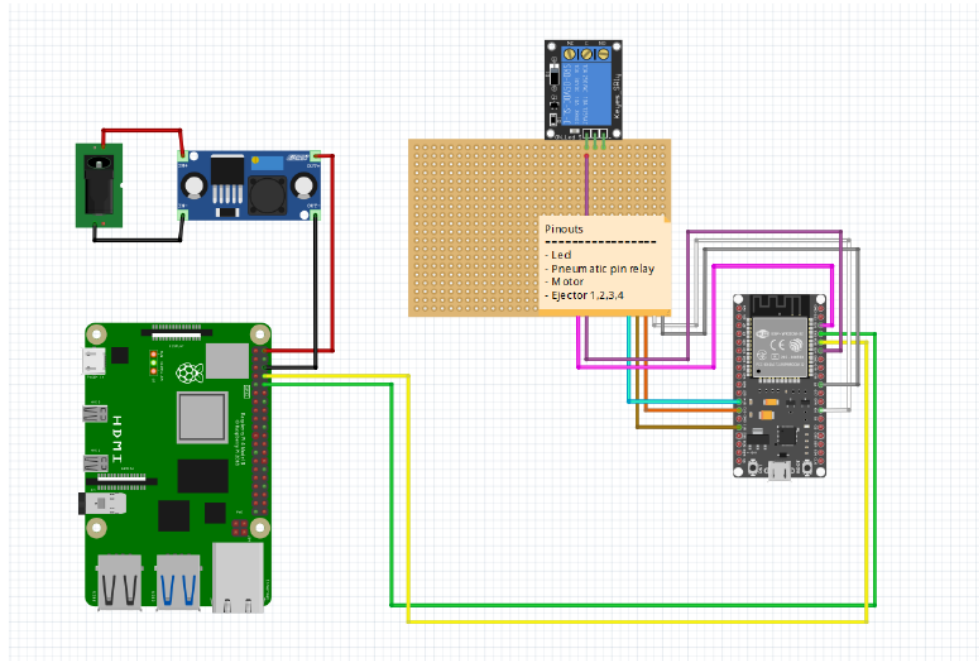


Figure A.3: Circuit implementation layout (Fritzing)

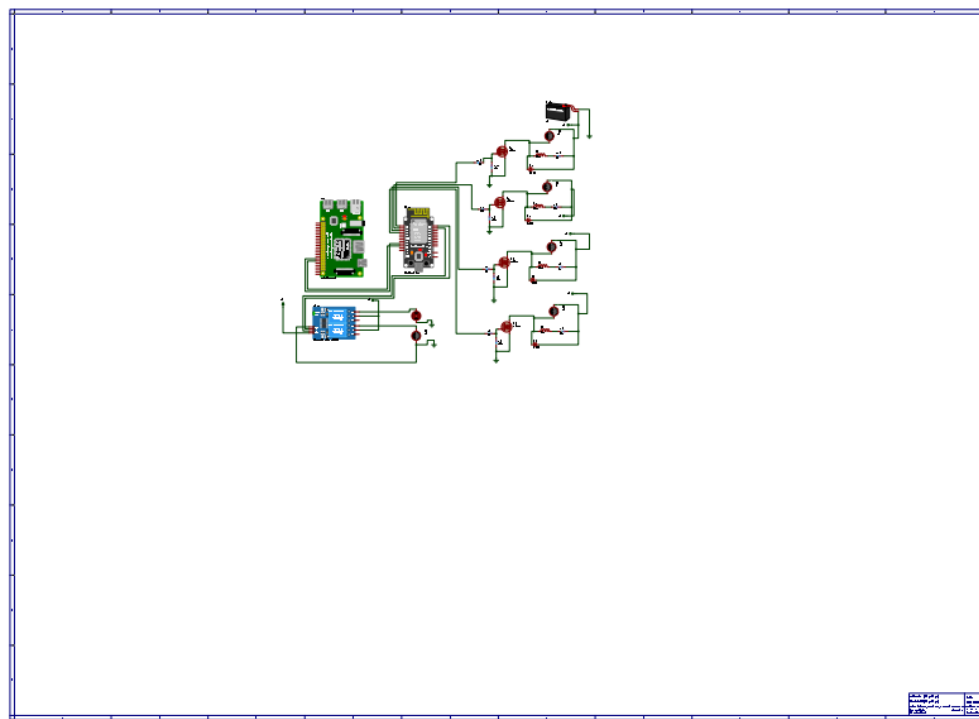


Figure A.4: Proteus full-circuit simulation

B Schedule and Budget

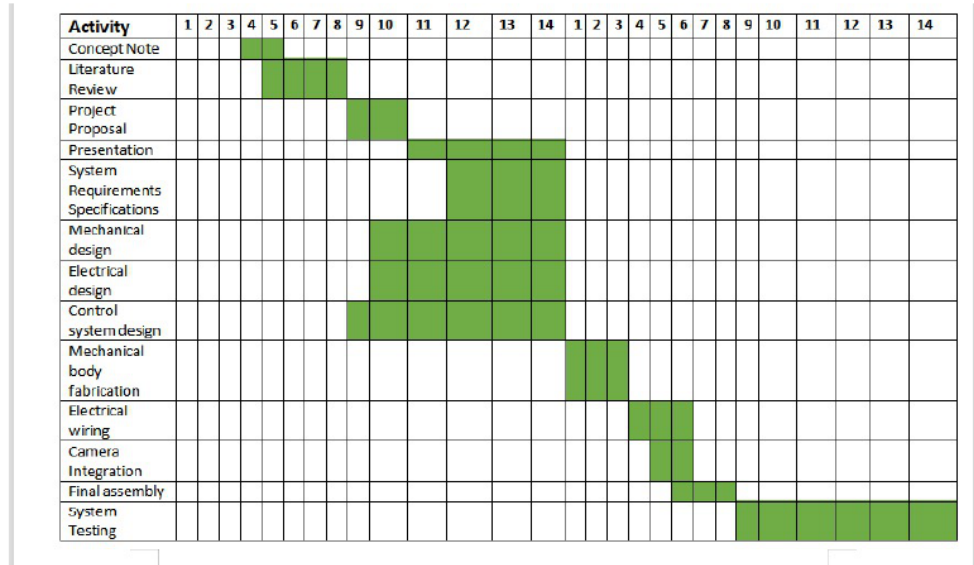


Figure B.1: Project schedule from the approved proposal

Component	Quantity	Price
Raspberry Pi 3	1	5500
Arduino 2560 Micro controller	1	3500
Camera	1	1500
Power Supply	1	3000
Air Ejectors	3	2500
Vibrating Motor	12v 200W (1)	1500
Relay	1	100
Miscellaneous		2000
TOTAL		19,600

Figure B.2: Project budget from the approved proposal

C Dataset and Model Outputs

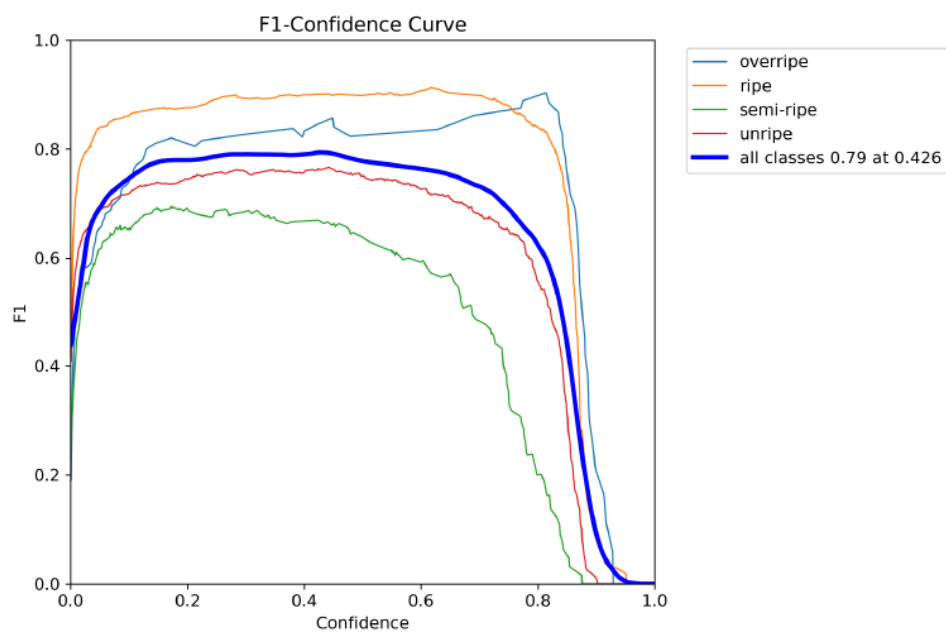


Figure C.1: Reference model F1 curve



Figure C.2: Reference model validation batch labels

D Mechanical Design Schematics

Detailed mechanical design schematics from the reference implementation [1] are provided below for reference. These technical drawings document the dimensional specifications and assembly details of the mechanical subsystem that is retained in the improved sorting machine.

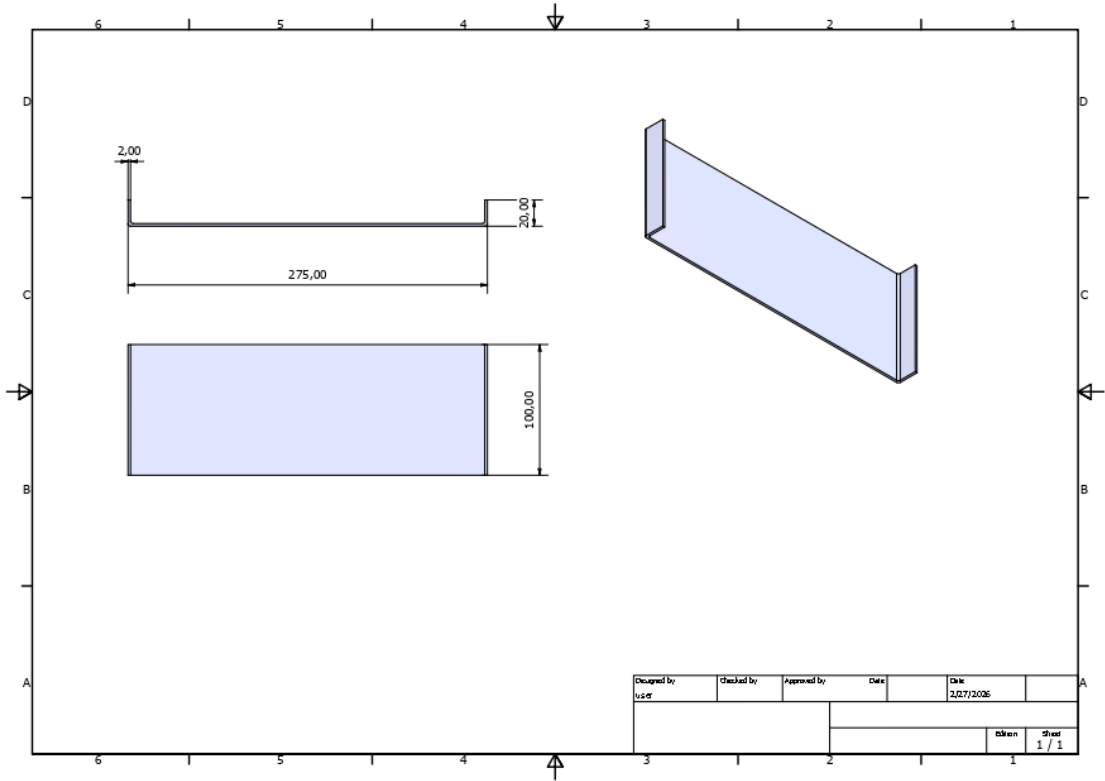


Figure D.1: Bottom plate technical drawing with dimensions and specifications

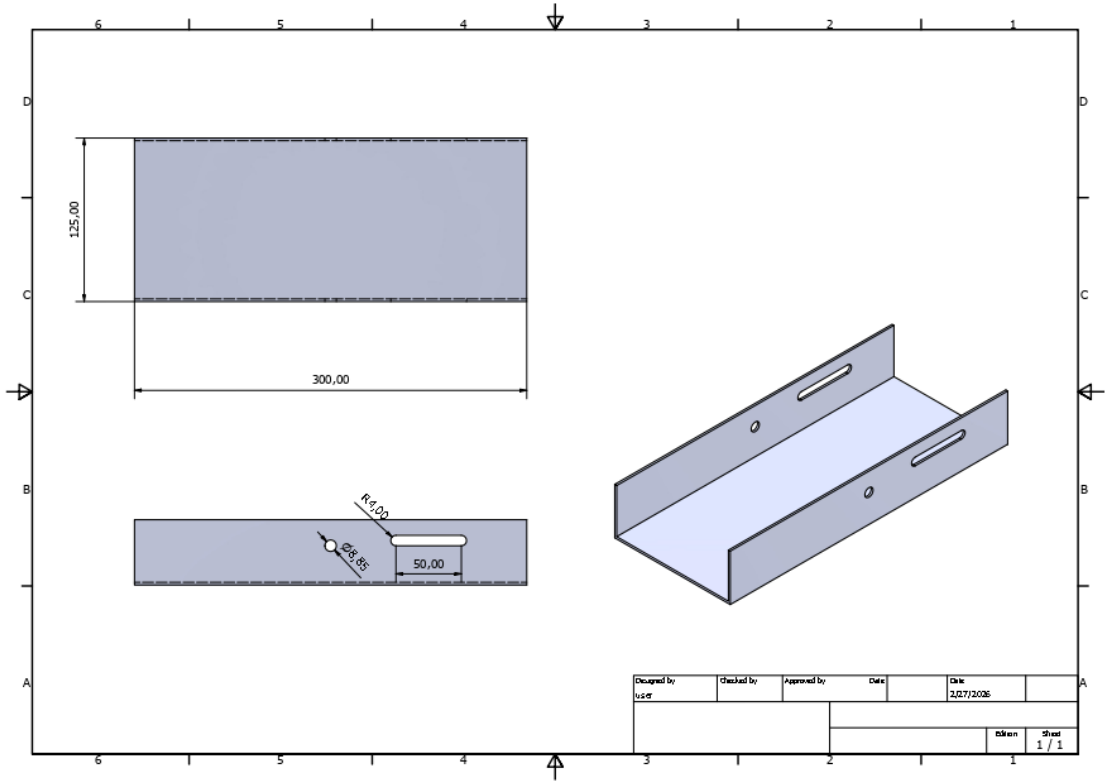


Figure D.2: Chute assembly technical drawing with dimensions and specifications

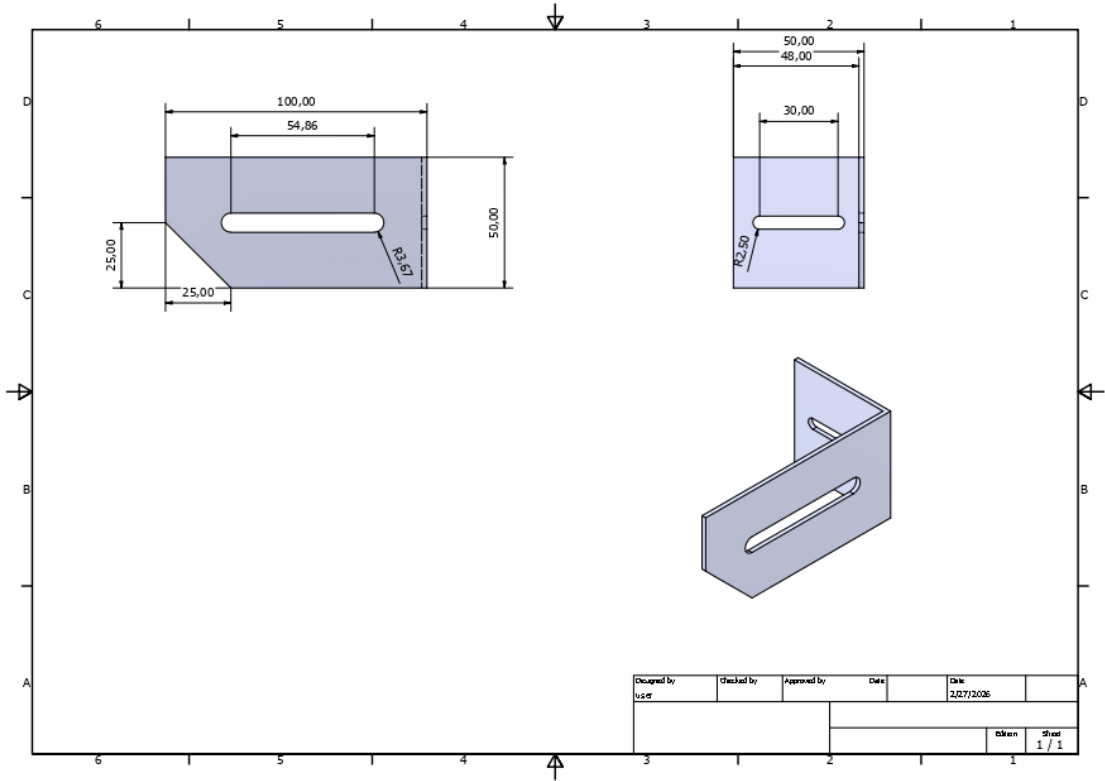


Figure D.3: Chute mounting bracket technical drawing with dimensions and specifications

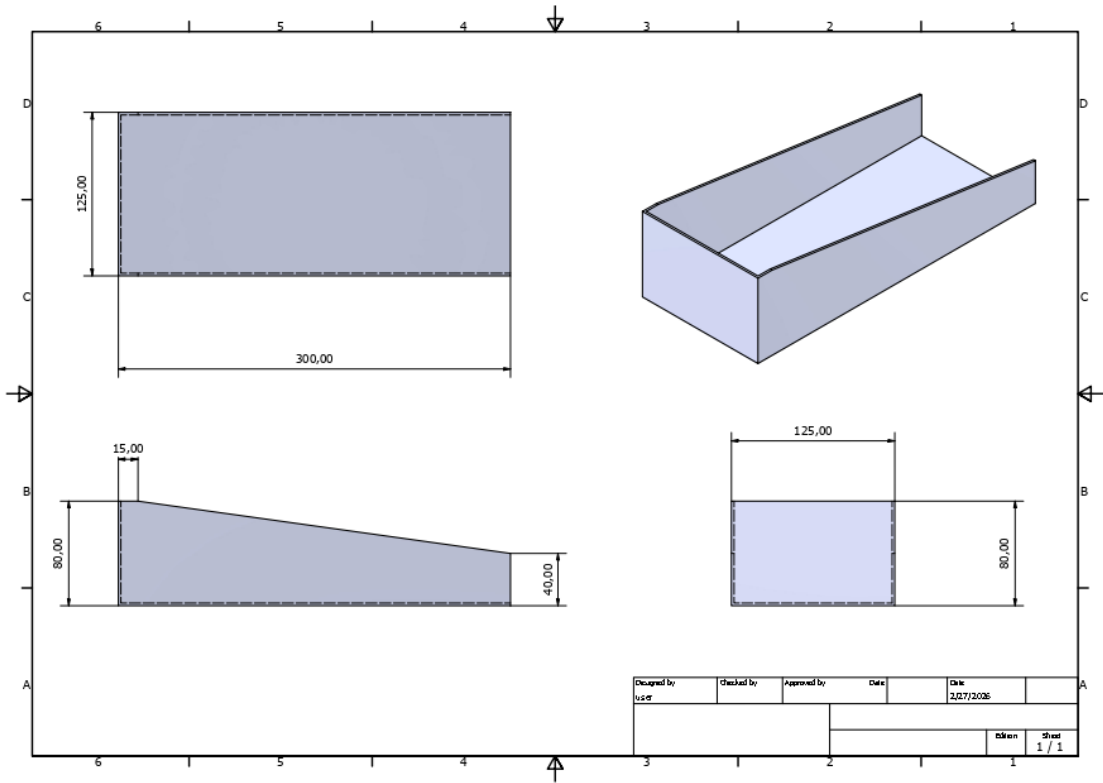


Figure D.4: Vibrating feeder trough technical drawing with dimensions and specifications

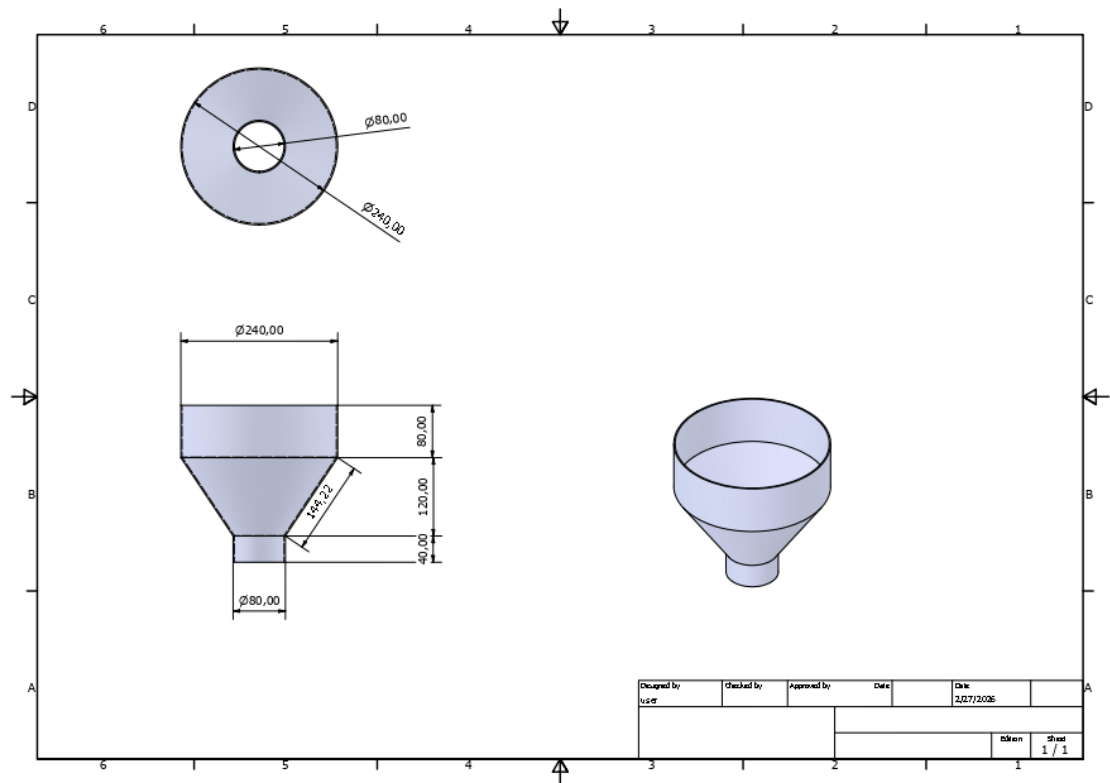


Figure D.5: Hopper assembly technical drawing with dimensions and specifications